

언어모델의 계산을 해부하다

선형대수에서 기계론적 해석가능성까지

LLM_Study

초판 연구 원고 · 2026년 8월 23일 기준

서문: 우리가 알고 있는 것과 아직 모르는 것

수학적 사실 또는 명시된 구현

가중치, 입력, 실행 코드, 수치 정밀도와 난수 상태가 고정되면 decoder-only Transformer의 순전파에서 계산되는 모든 텐서는 원칙적으로 재현할 수 있다. 그러나 그 텐서 계산이 학습한 인간 이해 가능한 특징, 회로, 알고리즘을 완전하게 기술하는 일반 이론은 아직 없다.

이 책의 목표는 이 두 문장을 동시에 이해하게 만드는 것이다. 첫 번째 문장은 Transformer를 신비화하지 않는다. 모델은 명시된 선형변환, 정규화, 비선형함수, attention, 잔차 덧셈을 실행한다. 두 번째 문장은 현재 과학의 경계를 과장하지 않는다. 행렬곱을 모두 기록하는 것과 그 계산을 설명하는 것은 동일하지 않다.

이 책의 중심 질문은 순전파의 정확한 계산과 학습된 메커니즘의 인과적 설명을 연결하는 것이다.

문자열이 입력된 순간부터 다음 토큰이 선택될 때까지 텐서의 모양과 값은 어떻게 변하는가? 그리고 학습된 계산에서 어떤 표현, 정보 흐름, 회로와 알고리즘을 인과적으로 확인할 수 있는가?

첫 질문에는 정확한 수식과 실행 가능한 코드로 답할 수 있다. 두 번째 질문에는 모델·과제·입력 분포·개입 방법을 제한한 실험적 답만 줄 수 있는 경우가 많다. 이 책은 두 종류의 답을 문장 수준에서 구분한다.

범위

주된 분석 대상은 자기회귀적 decoder-only Transformer이다. 특히 공개 논문과 구현을 함께 확인할 수 있는 GPT 계열, Llama 계열과 소형 연구 모델을 중심으로 한다. 원래의 encoder-decoder Transformer는 역사와 수식의 출발점으로 다루되, 현대 언어 생성에 필요한 causal masking, pre-normalization, RMSNorm, RoPE, SwiGLU, GQA, KV cache, prefill/decode를 별도로 전개한다.

훈련은 순전파가 왜 그런 가중치를 갖게 되는지 이해하는 데 필요한 범위에서 다룬다. next-token likelihood, 역전파, AdamW, scaling law와 대표적인 post-training 목표를 설명한다. 분산 학습 시스템, 데이터 거버넌스, 멀티모달 모델, 검색·도구 사용 시스템은 핵심 범위 밖이다.

이 책이 약속하지 않는 것

이 원고는 “모든 LLM이 같은 내부 알고리즘을 사용한다”고 주장하지 않는다. 특정 회로 연구의 결론을 그 연구가 조사하지 않은 모델과 입력으로 확대하지 않는다. 또한 attention heatmap, 높은 probe 정확도, 성공한 activation steering 가운데 어느 것도 그 자체로 완전한 기계론적 설명이라고 부르지 않는다.

출판 가능한 원고라는 목표는 오류가 없다는 선언이 아니라 다음의 검증 가능성을 뜻한다.

- 독자가 수식의 차원과 연산을 독립적으로 확인할 수 있다.
- 경험적 주장에는 원 연구와 적용 범위가 붙는다.
- 관찰, 인과적 증거, 해석과 열린 문제를 구분한다.
- 공개 모델에서 핵심 실험을 재현할 수 있는 절차가 있다.
- 기준일 이후의 연구로 수정될 수 있는 부분을 명시한다.

읽는 방법

Phase 1-3은 계산을 이해하는 구간이다. 이 구간에서는 수식을 손으로 유도하고 작은 텐서로 직접 계산해야 한다. Phase 4는 내부 상태에서 정보를 읽는 방법과 그 한계를 배운다. Phase 5-6은 구성요소를 회로로 묶고 인과적 개입으로 가설을 시험하는 구간이다. Phase 7은 이 도구를 *factual recall*, *in-context learning*, *reasoning*, *refusal*, *hallucination* 같은 행동에 적용한 연구를 비판적으로 읽는다.

각 Phase의 마지막 학습 점검은 단순 복습 문제가 아니다. 정의를 말할 수 있는지보다 수식, 텐서, 코드, 개입 결과를 서로 번역할 수 있는지를 검사한다. 이 번역 능력이 내부 메커니즘 연구의 실제 기초다.

판본과 갱신

이 원고의 문헌 기준일은 2026년 8월 23일이다. 빠르게 변하는 전면부 연구는 날짜와 모델 버전을 함께 기록한다. *peer review*를 거치지 않은 *preprint*와 연구기관의 기술 보고서는 그 상태를 숨기지 않는다. 새로운 반증이나 더 강한 재현 결과가 나오면 본문의 서술 강도도 함께 낮추거나 높여야 한다.

증거와 표현의 규칙

수학적 사실 또는 명시된 구현

“활성값에 정보가 존재한다”, “모델이 그 정보를 사용한다”, “특정 구성요소가 그 정보를 계산한다”는 서로 다른 주장이다. 첫째는 decodability, 둘째는 causal relevance, 셋째는 mechanism에 관한 주장이다. 한 종류의 증거만으로 나머지를 자동으로 결론낼 수 없다.

네 가지 서술 등급

본문의 색상 상자는 주장 종류를 표시한다.

수학적 사실 또는 명시된 구현

정의에서 유도되는 등식, 증명된 명제, 또는 특정 공개 코드가 실제로 수행하는 연산이다. 구현에 관한 문장은 모델·버전·코드 경로를 제한한다.

실험 결과

인용한 연구가 정한 모델, 데이터, metric과 intervention 아래에서 관찰한 결과다. 표본 밖의 모델이나 과제로 일반화하지 않는다.

제한된 해석

관찰을 압축하는 유용한 모델이나 연구자들의 해석이지만 유일한 설명으로 증명되지 않은 내용이다. “attention은 정보를 가져온다”, “MLP는 memory다” 같은 표현은 이 등급에서만 사용한다.

한계 또는 열린 문제

현재 방법으로 식별되지 않거나 논문 간 결론이 충돌하는 부분이다. 빈칸을 그럴듯한 이야기로 채우지 않는다.

설명의 다섯 수준

완전한 설명은 최소한 이 수준들을 연결해야 한다. 그러나 세부 수치 계산을 그대로 나열하면 완전하지만 이해하기 어렵고, 짧은 의사코드는 이해하기 쉽지만 실제 계산을 놓칠 수 있다. 따라서 “완전성”과 “압축성”은 별개의 평가축이다.

경험적 주장에 필요한 여섯 항목

경험적 주장은 가능한 경우 다음 여섯 정보를 같은 문단이나 인접 표에 기록한다.

1. 모델 이름, 크기, checkpoint와 architecture;
2. 입력 분포와 표본 선택 규칙;
3. 설명하려는 behavior와 평가 metric;

표 1: LLM을 설명한다는 말의 서로 다른 수준

수준	질문	검증 기준
구현	어떤 연산을 실행하는가?	코드와 텐서 값을 재현한다.
수학	입력에서 출력까지 함수는 무엇인가?	수식의 차원과 등식이 맞는다.
표현	어떤 변수가 내부 상태에서 복원되는가?	held-out data와 적절한 control을 사용한다.
인과	어떤 구성요소가 행동에 영향을 주는가?	명시된 intervention과 metric으로 효과를 재현한다.
알고리즘	구성요소가 어떤 절차로 협력하는가?	회로가 행동을 충실히 재현하고 반사실적 예측을 맞힌다.

4. 관찰하거나 개입한 activation, edge, weight 또는 feature;

5. clean, corrupted, patched 조건의 정확한 차이;

6. 결론이 적용되는 범위와 알려진 대안 설명.

특히 “중요하다”는 말은 metric을 생략하면 불완전하다. 정답 logit, logit difference, probability, KL divergence, cross-entropy, accuracy는 서로 다른 양이며 intervention의 순위도 바꿀 수 있다 (Zhang and Nanda, 2024; Miller et al., 2024).

이 책에서 금지하는 추론

- attention weight가 크다는 사실만으로 그 source token이 출력의 원인이라고 쓰지 않는다.
- linear probe가 성공했다는 사실만으로 원 모델이 그 방향을 사용한다고 쓰지 않는다.
- 한 위치를 편집할 수 있다는 사실만으로 지식이 오직 그곳에 저장됐다고 쓰지 않는다.
- ablation 효과가 작다는 사실만으로 구성요소가 불필요하다고 쓰지 않는다. downstream self-repair가 효과를 보상할 수 있다.
- 생성된 chain-of-thought를 내부 계산의 충실한 transcript라고 가정하지 않는다.
- 소형·합성 과제의 회로를 자연어 전반의 보편 알고리즘이라고 쓰지 않는다.

이 주의들은 단순한 철학적 경고가 아니다. probing control (Hewitt and Liang, 2019), attention 설명 논쟁 (Jain and Wallace, 2019; Wiegrefe and Pinter, 2019), activation patching의 방법 민감도 (Zhang and Nanda, 2024), self-repair (McGrath et al., 2023), localization과 editing의 불일치 (Hase et al., 2023), CoT 비충실성 (Turpin et al., 2023)에서 각각 실험적으로 문제가 드러났다.

출처 정책

핵심 주장은 원 논문, 공식 모델 보고서, 공개 구현 또는 표준 교재를 우선한다. survey는 문헌 지도를 찾는 데 사용할 수 있지만 원 결과의 대체 인용으로 쓰지 않는다. 기업 연구팀이 자기

모델을 분석한 기술 보고서는 **primary evidence**이지만 독립 재현이 아니라는 점을 유지한다. arXiv preprint는 동료심사가 완료된 논문과 동일하게 표시하지 않는다.

한계 또는 열린 문제

어떤 문헌 정책도 오류 가능성을 0으로 만들지는 않는다. 정확성은 출처의 권위만으로 보장되지 않으며, 수식 계산, 코드 대조, 재현, 반대 결과 확인과 판본 갱신이 필요하다.

Phase 1-7 학습 지도

수학적 사실 또는 명시된 구현

학습 순서는 “텐서를 계산할 수 있음”에서 “내부 알고리즘 가설을 인과적으로 시험할 수 있음”으로 진행한다. 뒤 Phase의 용어를 먼저 외우는 대신, 앞 Phase의 산출물을 다음 Phase의 측정 도구로 사용한다.

표 2: 각 Phase의 핵심 질문과 통과 기준

Phase	핵심 질문	통과 기준
1	벡터와 activation은 무엇인가?	affine map, dot product, norm, softmax, embedding을 좌표와 기저의 언어로 설명하고 작은 예를 손으로 계산한다.
2	한 번의 순전파에서 무슨 일이 생기는가?	embedding부터 logits까지 모든 텐서의 shape와 index를 적고 attention 한 행과 MLP update를 직접 계산한다.
3	현대 LLM은 2017 Transformer와 무엇이 다른가?	causal decoder의 RMSNorm, RoPE, SwiGLU, GQA, KV cache와 prefill/decode를 구현 수준에서 설명한다.
4	hidden state에서 무엇을 읽을 수 있는가?	probe, lens, geometry와 SAE가 측정하는 양을 구분하고 decodability와 causal use를 혼동하지 않는다.
5	구성요소는 어떻게 회로를 이루는가?	residual stream, QK/OV, head composition, MLP feature를 이용해 induction 또는 IOI 회로의 경로를 추적한다.
6	회로 가설을 어떻게 반증하는가?	ablation, activation/path patching, interchange intervention의 estimand와 통제 조건을 명시한다.
7	실제 행동의 메커니즘을 어디까지 아는가?	factual recall, ICL, reasoning, refusal, hallucination 연구의 증거와 적용 범위를 비교하고 미해결 부분을 분리한다.

두 개의 누적 프로젝트

프로젝트 A: 순전파를 직접 소유하기

Phase 1-3 동안 외부 Transformer 모듈을 호출하지 않는 최소 causal LM을 구현한다. 최종 코드는 적어도 다음 객체를 포함해야 한다.

```
class RMSNorm
class RotaryEmbedding
class CausalSelfAttention
class SwiGLU
class TransformerBlock
class CausalLM
```

프로젝트 A의 통과 기준은 각 원소가 왜 그 값을 갖는지 설명할 수 있는가이다. 이를 검증하려면 각 연산에서 batch, sequence, head, feature 축을 출력하고, 작은 입력에서는 수작업 계산과 일치하는 unit test를 작성해야 한다.

프로젝트 B: 회로 가설을 시험하기

프로젝트 B는 Phase 4-7 동안 공개된 소형 pretrained 모델에서 다음 검증 순서를 반복한다.

1. 관찰할 behavior와 clean/corrupted prompt 분포를 고정한다.
2. logits와 activation을 기록하고 descriptive hypothesis를 세운다.
3. probe나 attribution으로 후보를 좁힌다.
4. activation 또는 path intervention으로 causal effect를 측정한다.
5. 발견한 회로의 faithfulness, completeness, minimality와 범위 밖 입력을 시험한다.
6. 실패 사례에 따라 가설을 수정한다.

권장 산출물

각 Phase를 마칠 때 다음 네 파일을 남긴다.

- 한 페이지의 정의와 수식 요약;
- 직접 유도한 계산 노트;
- seed와 환경을 기록한 최소 재현 코드;
- 이해한 내용, 반례, 열린 질문을 분리한 연구 기록.

이 책은 정답 모음이 아니라 이러한 산출물을 만들기 위한 기준 문서다. 최종 목표는 논문의 용어를 알아보는 것이 아니라, 논문의 intervention이 실제로 어느 tensor를 어떻게 바꾸며 그 결과가 어떤 인과 주장을 지지하는지 독립적으로 판단하는 것이다.

차례

서문: 우리가 알고 있는 것과 아직 모르는 것	ii
증거와 표현의 규칙	iv
Phase 1–7 학습 지도	vii
I Phase 1 · 신경망 표현의 수학	1
1 벡터에서 신경망 표현까지	2
1.1 결론부터: hidden state는 좌표를 가진 벡터다	2
1.2 벡터공간, 기저와 좌표	2
1.2.1 tensor와 축	3
1.3 행렬은 선형사상이다	3
1.3.1 rank, kernel과 정보 손실	3
1.4 내적, norm, 각도와 cosine similarity	3
1.5 고윳값, SVD와 저차원 구조	4
1.6 확률분포, logits와 softmax	5
1.7 미분, 계산 그래프와 역전파	5
1.8 embedding, feature, activation, representation	6
1.9 Phase 1 학습 점검	7
II Phase 2 · Transformer 순전파	8
2 Decoder-only Transformer의 정확한 순전파	9
2.1 결론부터: 다음 토큰 분포는 합성함수의 출력이다	9
2.2 기호와 tensor shape	9
2.3 토큰 ID에서 residual stream으로	9
2.4 한 attention head의 모든 계산	10
2.5 multi-head attention과 output projection	11
2.6 MLP update와 잔차 덧셈	12
2.7 마지막 정규화, unembedding과 logits	12
2.8 autoregressive generation	13
2.9 완전한 shape 추적 예제	13
2.10 Phase 2 학습 점검	13

III Phase 3 · 현대 decoder-only LLM	15
3 현대 LLM의 구성요소와 실행 경로	16
3.1 결론부터: 현대 모델은 하나의 고정 설계가 아니다	16
3.2 decoder-only와 causal attention	16
3.3 pre-normalization과 RMSNorm	17
3.4 RoPE	17
3.5 SwiGLU	18
3.6 MHA, MQA와 GQA	18
3.7 tokenization과 vocabulary	19
3.8 KV cache, prefill과 decode	19
3.9 FlashAttention과 수치적 동등성	20
3.10 weight tying과 LM head	21
3.11 훈련이 가중치를 만드는 과정	21
3.12 Phase 3 학습 점검	22
IV Phase 4 · 표현을 관찰하는 방법	23
4 내부 표현을 읽는 방법과 그 한계	24
4.1 결론부터: 읽을 수 있다는 사실은 사용된다는 뜻이 아니다	24
4.2 분산 표현과 특징 방향	24
4.3 표현 공간의 기하	25
4.4 선형 probe와 control task	25
4.5 logit lens와 tuned lens	26
4.6 중첩과 다의적 neuron	26
4.7 희소 오토인코더	27
4.8 표현 분석의 최소 증거 기준	27
4.9 Phase 4 학습 점검	28
V Phase 5 · 회로와 내부 알고리즘	29
5 잔차 스트림, 회로와 내부 알고리즘	30
5.1 결론부터: 회로는 행동에 상대적인 계산 부분그래프다	30
5.2 residual stream을 통신 채널로 보기	30
5.3 direct logit attribution과 경로 전개	31
5.4 QK 회로와 OV 회로	31
5.5 head composition	32
5.6 induction head	32
5.7 IOI 회로	33
5.8 MLP feature와 memory 해석	33
5.9 완전한 알고리즘을 아는 특수 사례	34
5.10 Phase 5 학습 점검	34

VI Phase 6 · 인과적 기계론적 해석	36
6 개입으로 메커니즘을 시험하기	37
6.1 결론부터: 위치 찾기와 메커니즘 설명은 다르다	37
6.2 관찰, 상관과 개입	37
6.3 ablation	37
6.4 activation patching과 causal tracing	38
6.5 path patching과 edge intervention	39
6.6 interchange intervention과 causal abstraction	39
6.7 attribution과 gradient 근사	39
6.8 자동 회로 발견	40
6.9 faithfulness, completeness, minimality	40
6.10 self-repair와 방법 민감도	41
6.11 Phase 6 학습 점검	41
VII Phase 7 · 실제 LLM 행동의 메커니즘	43
7 실제 LLM 행동에서 발견된 메커니즘	44
7.1 결론부터: 발견은 구체적이며 보편성은 미확립이다	44
7.2 factual recall	44
7.3 factual recall의 세 단계: 원 연구의 정확한 주장	44
7.3.1 2510.09033의 Section 3.3은 무엇을 다시 정의하는가	45
7.4 hallucination과 내부 상태	46
7.5 in-context learning과 copying	46
7.6 reasoning과 chain-of-thought	47
7.7 entity tracking, planning과 refusal	47
7.8 대형 모델의 attribution graph	47
7.9 모델 간 일반화 문제	48
7.10 Phase 7 학습 점검	49
VIII 현재의 경계와 연구 프로그램	50
8 완전한 설명은 어디까지 왔는가	51
8.1 현재의 결론	51
8.2 완전성, 충실성, 압축성과 확장성	51
8.3 알려진 것의 경계	52
8.4 연구 프로그램	52
8.5 종료 조건이 아니라 갱신 규칙	53
IX 부록	54
A 기호와 용어의 기준	55
A.1 기호를 고정하는 이유	55

A.2	측과 index의 방향	56
A.3	한국어와 원어 용어	56
A.4	인식론적 동사의 강도	57
B	핵심 수식 유도	58
B.1	softmax의 shift invariance와 안정적 계산	58
B.2	cross-entropy와 logit gradient	58
B.3	scaled dot-product attention의 차원과 scale	59
B.4	여러 head의 출력은 residual update의 합이다	59
B.5	RoPE가 상대 offset을 포함하는 이유	59
B.6	RMSNorm이 보존하고 바꾸는 양	60
B.7	KV cache가 같은 attention을 계산하는 이유	60
B.8	GQA가 줄이는 parameter와 cache	60
B.9	direct logit attribution과 final norm	61
B.10	activation patching의 first-order 근사	61
B.11	Jensen-Shannon divergence	61
C	재현 실험 프로토콜	63
C.1	재현의 최소 단위	63
C.2	Lab 1: 손으로 계산한 attention과 코드의 일치	63
C.3	Lab 2: 최소 decoder-only Transformer 구현	64
C.4	Lab 3: 공개 reference implementation과 대조	64
C.5	Lab 4: prefill과 KV-cache decode의 동등성	64
C.6	Lab 5: probe는 정보 존재만 보고한다	65
C.7	Lab 6: induction circuit 재현	65
C.8	Lab 7: activation patching의 방법 민감도	65
C.9	Lab 8: 사실 회상의 세 단계 재현	65
C.10	Lab 9: 2510.09033의 분류와 검출 결과 감사	66
C.11	Lab 10: ground-truth circuit에서 방법 평가	66
C.12	실험 중단과 실패 보고 규칙	66
D	주장-증거 감사표	67
D.1	감사표의 목적	67
D.2	갱신 절차	69
	찾아보기	76

그림 목차

5.1	한 block에서 residual stream을 읽고 update를 더하는 경로	31
7.1	Geva et al. (2023)이 제안한 사실 회상의 세 단계	45

표 목차

1	LLM을 설명한다는 말의 서로 다른 수준	v
2	각 Phase의 핵심 질문과 통과 기준	vii
2.1	기본 차원과 의미	10
2.2	한 block과 LM head의 shape 추적	14
3.1	prefill과 한 token decode의 계산 경로	20
4.1	표현에 관한 주장과 필요한 추가 증거	28
8.1	현대 decoder-only LLM에 대해 알려진 것과 남은 문제	52
A.1	핵심 기호와 차원	55
A.2	권장 용어와 이 책에서의 의미	56
D.1	핵심 주장과 증거 범위	67

제 I 부

Phase 1 · 신경망 표현의 수학

1 벡터에서 신경망 표현까지

1.1 결론부터: hidden state는 좌표를 가진 벡터다

특정 layer ℓ 과 token position t 의 hidden state를 $\mathbf{h}_t^{(\ell)} \in R^{d_{\text{model}}}$ 로 쓸 수 있다. batch와 모든 위치를 함께 모으면

$$\mathbf{X}^{(\ell)} \in R^{B \times T \times d_{\text{model}}} \quad (1.1)$$

이다. 이것은 비밀 정보가 들어 있는 별도 저장장치가 아니라 그 지점까지의 계산 결과다. 모델 구현에서 “hidden states”는 보통 여러 layer의 이런 activation을 뜻하며, 기계론적 분석에서는 residual branch들이 더해지는 공용 상태라는 점을 강조해 “residual stream”이라고 부른다.

수학적 사실 또는 명시된 구현

벡터는 기저를 정해야 숫자 좌표로 표시된다. 따라서 hidden state의 i 번째 좌표가 항상 독립된 하나의 인간 개념이라는 결론은 선형대수에서 나오지 않는다. 같은 추상 벡터는 가역적인 기저변환 아래 다른 좌표를 갖는다.

이 Phase의 목적은 벡터를 단순한 숫자 목록으로 보지 않고, 선형사상과 비선형사상이 정보를 표현하고 변환하는 공간으로 보는 것이다. 필요한 수학의 범위는 [Deisenroth et al. \(2020\)](#)의 선형대수·해석기하·벡터미적분과 대응한다.

1.2 벡터공간, 기저와 좌표

실수 벡터공간 V 는 벡터 덧셈과 scalar multiplication에 대해 닫혀 있고 해당 연산의 공리를 만족하는 집합이다. R^d 는 표준 예다. 벡터 $\mathbf{x} \in R^d$ 는 표준기저 $\mathbf{e}_1, \dots, \mathbf{e}_d$ 에 대해

$$\mathbf{x} = \sum_{i=1}^d x_i \mathbf{e}_i \quad (1.2)$$

로 쓴다. x_i 는 벡터 자체가 아니라 선택한 기저에서의 좌표다.

선형독립인 벡터들의 집합이 공간 전체를 span하면 기저다. 벡터들 $\mathbf{v}_1, \dots, \mathbf{v}_k$ 의 span은

$$\text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\} = \left\{ \sum_{i=1}^k a_i \mathbf{v}_i : a_i \in R \right\} \quad (1.3)$$

이다. 표현 연구에서 “어떤 정보가 subspace에 있다”는 문장은 보통 후보 방향들이 span하는 부분공간에서 그 정보를 decode하거나 조작할 수 있다는 뜻이다. 어떤 부분공간이 의미론적으로 유일한지는 별도의 식별 문제다.

일관된 기저변환은 함수는 보존하지만 개별 좌표의 값을 바꾼다. 기저를 $\mathbf{S}$ 로 바꿔 좌표를 $\tilde{\mathbf{x}} = \mathbf{S}^{-1}\mathbf{x}$ 로 쓰면, 동일한 선형사상 $\mathbf{W}$ 의 좌표표현은 $\tilde{\mathbf{W}} = \mathbf{S}^{-1}\mathbf{W}\mathbf{S}$ 가 된다.

1.2.1 tensor와 축

tensor는 문맥에 따라 다중선형대수의 객체 또는 다차원 수치 배열을 뜻한다. 딥러닝 구현에서는 후자의 뜻이 흔하다. shape $[B, T, d]$ 는 각각 batch, sequence, model-feature 축을 뜻한다. 축 이름을 생략하면 같은 숫자 배열이라도 어떤 contraction을 해야 하는지 알 수 없다.

token별 선형변환은 row-vector 구현 관례에서 다음 contraction으로 계산된다.

$$\mathbf{Y}_{b,t,:} = \mathbf{X}_{b,t,:} \mathbf{W}, \quad \mathbf{X} \in R^{B \times T \times d_{\text{in}}}, \quad \mathbf{W} \in R^{d_{\text{in}} \times d_{\text{out}}} \quad (1.4)$$

로 계산되며 $\mathbf{Y} \in R^{B \times T \times d_{\text{out}}}$ 다. 논문이 column vector 관례를 쓰면 $\mathbf{y} = \mathbf{W}\mathbf{x}$ 이고 가중치 shape가 전치된다. 두 표기법은 모순이 아니지만 한 유도 안에서는 섞으면 안 된다.

1.3 행렬은 선형사상이다

선형사상 $f: V \rightarrow U$ 는 $f(a\mathbf{x} + b\mathbf{y}) = af(\mathbf{x}) + bf(\mathbf{y})$ 를 만족한다. 기저를 고르면 행렬 $\mathbf{W}$ 로 표현된다. column-vector 관례에서

$$\mathbf{y} = \mathbf{W}\mathbf{x}, \quad \mathbf{W} \in R^{m \times d}, \quad \mathbf{x} \in R^d, \quad \mathbf{y} \in R^m. \quad (1.5)$$

행렬의 i 번째 row와 $\mathbf{x}$ 의 내적이 출력 y_i 다. 반대로 각 column은 해당 입력 좌표가 출력 공간에 기여하는 방향이다.

$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ 는 affine map이다. bias가 0이 아니면 원점을 보존하지 않으므로 엄밀히 선형은 아니지만 신경망 문헌에서는 흔히 “linear layer”라고 부른다. embedding lookup도 one-hot vector $\mathbf{e}_v$ 에 행렬을 곱하는 선형연산으로 쓸 수 있다.

1.3.1 rank, kernel과 정보 손실

$\text{rank}(\mathbf{W})$ 는 column space의 차원이다. $\text{rank}(\mathbf{W}) < d$ 이면 서로 다른 입력이 같은 출력으로 갈 수 있다. kernel은

$$\ker(\mathbf{W}) = \{\mathbf{x} : \mathbf{W}\mathbf{x} = \mathbf{0}\} \quad (1.6)$$

이며, 여기에 속하는 변화는 해당 선형사상 뒤에서 관찰되지 않는다. attention head의 projection이 보통 d_{model} 보다 작은 d_{head} 로 향한다는 사실은 한 head가 residual space 전체를 독립적으로 읽거나 쓰지 못한다는 정확한 rank 제약을 준다.

선형사상의 합과 합성은 구분해야 한다.

$$(\mathbf{A} + \mathbf{B})\mathbf{x} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{x}, \quad (\mathbf{AB})\mathbf{x} = \mathbf{A}(\mathbf{B}\mathbf{x}). \quad (1.7)$$

잔차 스트림은 여러 update의 합을 만들고, 회로의 경로는 여러 projection의 합성을 만든다. 이 차이가 이후 direct path와 composed path를 분해하는 기초가 된다.

1.4 내적, norm, 각도와 cosine similarity

표준 Euclidean 내적과 L_2 norm은

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y}, \quad \|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{x}} \quad (1.8)$$

이다. 내적은 projection과 유사도를 수량화하며 attention score의 핵심 연산이다. cosine similarity는 영벡터가 아닌 두 벡터에 대해

$$\cos(x, y) = \frac{x^T y}{\|x\|_2 \|y\|_2} \quad (1.9)$$

로 정의되고 크기를 제거한 방향의 일치를 측정한다.

제한된 해석 · 기하학적 측정의 범위

cosine similarity가 높으면 선택한 내적 아래 두 좌표 벡터의 각도가 작다는 뜻이다. 그 자체로 두 activation이 같은 의미, 같은 원인, 같은 기능을 갖는다고 증명하지 않는다. 표현 공간의 내적 선택도 의미론과 무관한 자동 선택은 아니다 (Park et al., 2024).

두 벡터가 직교한다는 것은 내적이 0이라는 뜻이지 확률적 독립이나 causal independence를 뜻하지 않는다. 반대로 고차원에서 여러 방향이 거의 직교할 수 있다는 사실은 feature를 dimension보다 많이 배치할 수 있는 superposition 직관의 일부지만, 실제 LLM이 특정 방식으로 superposition을 사용한다는 결론은 실험이 필요하다.

1.5 고윳값, SVD와 저차원 구조

정사각행렬 A 의 고유벡터 $v \neq 0$ 와 고윳값 λ 는 $Av = \lambda v$ 를 만족한다. 이는 A 가 특정 방향을 회전시키지 않고 scale만 바꾸는 경우를 기술한다. 모든 행렬이 실수 고유기저로 대각화되는 것은 아니다.

반면 임의의 실수행렬 $W \in R^{m \times d}$ 에는 singular value decomposition이 존재한다.

$$W = U \Sigma V^T. \quad (1.10)$$

V 의 right singular vectors는 입력 방향, U 의 left singular vectors는 대응하는 출력 방향, diagonal의 singular values는 증폭률이다. 상위 r 개 항만 남긴 W_r 은 Frobenius norm과 spectral norm에서 최적의 rank- r 근사다.

SVD는 다음 이유로 내부 분석에 직접 쓰인다.

- projection과 weight product의 유효 rank를 확인한다.
- residual activation의 principal directions를 찾는다.
- attention head의 W_{OV} 와 W_{QK} 가 읽고 쓰는 저차원 subspace를 요약한다.
- 해석용 저차원 근사가 원 함수를 얼마나 잃는지 singular spectrum으로 측정한다.

한계 또는 열린 문제

큰 singular value가 자동으로 의미 있는 feature를 뜻하지 않는다. SVD는 variance나 operator gain에 관한 최적성을 주지만 인간 개념, sparsity 또는 causal relevance를 목적으로 최적화하지 않는다.

1.6 확률분포, logits와 softmax

vocabulary 크기를 V 라 하면 LM head는 마지막 표현을 $\mathbf{z} \in \mathbb{R}^V$ 로 보낸다. z_i 는 token i 의 logit이다. softmax는

$$p_i = \text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{j=1}^V \exp(z_j)} \quad (1.11)$$

로 확률 simplex의 점을 만든다. 모든 logit에 같은 상수 c 를 더해봐도 분포는 변하지 않는다. 수치 구현은 overflow를 피하려고 보통 $m = \max_j z_j$ 를 빼서

$$p_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)} \quad (1.12)$$

를 계산한다.

temperature $\tau > 0$ 를 사용하면 $p_i(\tau) = \text{softmax}(\mathbf{z}/\tau)_i$ 다. τ 는 모델의 가중치나 logits 생성 계산을 바꾸지 않고 선택 전 분포를 바꾼다. $\tau \rightarrow 0^+$ 이면 최대 logit에 질량이 집중되고, 큰 τ 에서는 분포가 평평해진다.

정답 token y 에 대한 cross-entropy 또는 negative log-likelihood는

$$L(\mathbf{z}, y) = -\log p_y = -z_y + \log \sum_j \exp z_j \quad (1.13)$$

이고 logit에 대한 gradient는

$$\frac{\partial L}{\partial z_i} = p_i - \mathbf{1}[i = y] \quad (1.14)$$

이다. 이 식은 정답 logit을 올리고 현재 확률에 비례해 다른 logits를 내리는 학습 신호를 정확히 보여준다.

수학적 사실 또는 명시된 구현

softmax 확률은 모델과 context가 정한 다음-token 조건부분포다. 세계의 명제가 참일 확률, 모델이 답을 “안다”는 확률, 또는 calibration이 보장된 신뢰도가 아니다. 이들 사이의 관계는 별도 데이터와 평가로 검증해야 한다.

1.7 미분, 계산 그래프와 역전파

gradient $\nabla_{\theta} L$ 은 각 parameter 좌표의 미소 변화에 대한 loss의 1차 변화를 모은다. 순전파가 $\mathbf{y} = f_{\theta}(\mathbf{x})$ 를 계산하고 scalar loss $L(\mathbf{y})$ 를 만든다고 하자. 합성함수 $f \circ g$ 의 Jacobian은 chain rule

$$\mathbf{J}_{f \circ g}(\mathbf{x}) = \mathbf{J}_f(g(\mathbf{x})) \mathbf{J}_g(\mathbf{x}) \quad (1.15)$$

을 따른다.

reverse-mode automatic differentiation은 출력 cotangent에서 시작해 계산 그래프를 역순으로 vector-Jacobian product를 누적한다. scalar loss와 매우 많은 parameter를 가진 신경망에 적합하다. backpropagation은 이 chain rule을 신경망 그래프에 효율적으로 적용한 경우이며, numerical differentiation이나 symbolic differentiation와 동일한 방법이 아니다 (Rumelhart et al., 1986; Baydin et al., 2018).

가장 단순한 gradient descent update는

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} L(\theta_k) \quad (1.16)$$

이다. 실제 LLM 훈련은 minibatch, momentum, adaptive moments, weight decay, learning-rate schedule, mixed precision과 분산 통신을 사용한다. 그러나 이런 optimizer가 만드는 것은 가중치 update이지 forward-pass 안의 token별 activation update와는 다르다.

구분해야 할 두 시간축

- **훈련 시간:** 여러 batch에 걸쳐 parameter θ 가 바뀐다.
- **추론 시간:** 고정된 θ 아래 token activation이 layer와 생성 step을 따라 바뀐다.

in-context learning에서 activation이 predictor 상태처럼 동작할 수 있어도, 일반적인 추론에서는 optimizer가 model parameter를 업데이트하는 것이 아니다.

1.8 embedding, feature, activation, representation

용어는 다음처럼 고정한다.

embedding

이 책에서는 우선 token ID를 d_{model} 차원 벡터로 보내는 learned lookup을 뜻한다. 문맥화된 후속 activation까지 넓게 embedding이라 부르는 문헌도 있으므로 인용할 때 정의를 확인한다.

activation

특정 입력에 대해 계산된 중간 수치다. residual vector, attention pattern, MLP pre-activation과 SAE coefficient가 모두 activation의 예다.

hidden state

출력으로 직접 관찰되지 않는 중간 상태라는 넓은 이름이다. Transformer에서는 보통 특정 layer 경계의 token별 residual vector를 가리키지만 API마다 반환 지점이 다르다.

representation

어떤 변수나 입력을 나타내는 내부 상태 또는 그 상태를 만드는 scheme이다. “표현한다”는 말은 단순 correlation, decodability, causal use 중 무엇을 뜻하는지 추가해야 한다.

feature

입력에서 의미 있는 속성으로 간주하는 변수다. 한 neuron, 한 좌표, 한 direction, 한 SAE unit과 동일하다고 정의되지 않는다. feature와 activation-space object의 alignment는 해석 가설의 일부다.

예제 1.1. embedding lookup은 one-hot 입력에 대한 선형변환으로 나타낼 수 있다. token ID v 의 one-hot row vector를 e_v^T 라 하고 embedding matrix를 $E \in R^{V \times d}$ 라 하면 $x_v^T = e_v^T E$ 는 E 의 v 번째 row를 선택한다.

1.9 Phase 1 학습 점검

학습 점검

다음은 외부 자료 없이 수행할 수 있어야 한다.

1. $[B, T, d]$ activation에 $[d, m]$ weight를 적용한 결과 shape를 적는다.
2. 2×2 행렬의 column space, kernel, rank를 계산한다.
3. 두 벡터의 dot product, norm, cosine similarity를 계산하고 각각의 의미를 구분한다.
4. 세 logits의 stable softmax와 정답 cross-entropy를 손으로 계산한다.
5. affine layer와 softmax-cross-entropy를 잇는 계산 그래프에서 gradient를 유도한다.
6. “probe가 정보를 읽었다”와 “모델이 정보를 사용했다”가 왜 다른 주장인지 설명한다.

한계 또는 열린 문제

Phase 1을 마쳐도 어떤 hidden direction이 실제 인간 개념에 대응하는지는 알 수 없다. 알게 되는 것은 후보 표현을 정확한 공간·함수·측정의 언어로 묻는 방법이다.

제 II 부

Phase 2 · Transformer 순전파

2 Decoder-only Transformer의 정확한 순전파

2.1 결론부터: 다음 토큰 분포는 합성함수의 출력이다

decoder-only Transformer의 한 번의 순전파는 token sequence를 vocabulary 전체의 logits로 변환하는 결정론적 함수다. dropout을 끄고 가중치와 수치 연산을 고정하면, 입력 token IDs가 같을 때 logits도 같다. 다음 token을 실제로 고르는 sampling 단계에는 별도의 난수가 들어갈 수 있다.

pre-normalization을 사용하는 순차형 block은 attention update와 MLP update를 차례로 residual stream에 더한다.

$$U^{(\ell)} = X^{(\ell)} + \text{Attn}^{(\ell)} \left(\text{Norm}_1^{(\ell)}(X^{(\ell)}) \right), \quad (2.1)$$

$$X^{(\ell+1)} = U^{(\ell)} + \text{MLP}^{(\ell)} \left(\text{Norm}_2^{(\ell)}(U^{(\ell)}) \right). \quad (2.2)$$

마지막 layer 뒤에서는

$$H = \text{Norm}_f(X^{(L)}), \quad (2.3)$$

$$Z = HW_U^T + b_U, \quad (2.4)$$

$$p(x_{T+1} = v \mid x_{1:T}) = \text{softmax}(Z_{:,T,:})_v \quad (2.5)$$

를 계산한다. 실제 구현은 bias를 생략하거나 embedding과 unembedding weight를 공유할 수 있다. 원래 Transformer의 모든 변형이 식 (2.2)과 같은 배치를 사용하는 것은 아니다 (Vaswani et al., 2017; Xiong et al., 2020).

2.2 기호와 tensor shape

순전파를 계산하려면 모든 수식에서 같은 축과 차원 기호를 사용해야 한다. 이 장에서는 다음 기호를 고정한다.

수식에서는 읽기 쉽도록 batch 축을 생략할 때가 있다. 그러나 실제 구현에서 batch 축이 사라지는 것은 아니다. position i 의 query는 destination, position j 의 key와 value는 source라고 부르겠다. 이 이름을 고정하면 attention matrix의 행과 열을 뒤바꾸는 실수를 줄일 수 있다.

2.3 토큰 ID에서 residual stream으로

tokenizer가 문자열을 정수열 $(t_1, \dots, t_T)$ 로 바꾸면 $t_i \in \{0, \dots, V-1\}$ 이다. embedding matrix $E \in R^{V \times d}$ 에서 각 row를 선택하여

$$X_{b,i,:}^{(0)} = E_{t_{b,i},:} \quad (2.6)$$

를 얻는다. absolute positional embedding을 사용하는 모델은 position vector를 여기서 더할 수 있다. RoPE를 사용하는 모델은 일반적으로 input embedding에 position vector를 더하지 않고 attention 안의 query와 key를 회전한다. 자세한 내용은 장 3에서 다룬다.

표 2.1: 기본 차원과 의미

기호	의미	대표 shape 또는 범위
B	batch에 포함된 sequence 수	양의 정수
T	현재 sequence 길이	양의 정수
V	vocabulary 크기	양의 정수
d	residual stream 차원, d_{model}	양의 정수
H	query 및 MHA의 head 수	양의 정수
d_h	한 head의 차원	보통 d/H
d_{ff}	MLP 중간 차원	모델별로 다름
L	Transformer block 수	양의 정수
$\mathbf{X}^{(\ell)}$	layer ℓ 입구의 residual stream	$[B, T, d]$
$\mathbf{Z}$	각 위치와 token의 logit	$[B, T, V]$

padding을 포함하는 batch에서는 padding 위치를 막는 mask가 causal mask와 별도로 필요할 수 있다. 반면 단일 sequence만 처리하며 padding이 없다면 causal mask만 있으면 된다. tokenizer가 만든 token 경계와 모델이 처리하는 position은 문자나 단어 경계와 일치할 필요가 없다.

수학적 사실 또는 명시된 구현

한 position의 초기 벡터는 그 token ID에 대응하는 embedding이다. 첫 attention layer를 통과한 뒤에는 앞선 여러 position의 value가 섞일 수 있으므로, 같은 token ID도 문맥에 따라 서로 다른 residual vector를 갖는다.

2.4 한 attention head의 모든 계산

한 attention head는 정규화한 residual stream에서 query, key와 value를 각각 투영한다. $\mathbf{R} = \text{Norm}(\mathbf{X}) \in R^{B \times T \times d}$ 라고 두면 head h 의 세 projection은 다음과 같다.

$$\mathbf{Q}^h = \mathbf{R}\mathbf{W}_Q^h, \quad \mathbf{W}_Q^h \in R^{d \times d_h}, \quad (2.7)$$

$$\mathbf{K}^h = \mathbf{R}\mathbf{W}_K^h, \quad \mathbf{W}_K^h \in R^{d \times d_h}, \quad (2.8)$$

$$\mathbf{V}^h = \mathbf{R}\mathbf{W}_V^h, \quad \mathbf{W}_V^h \in R^{d \times d_h}. \quad (2.9)$$

따라서 $\mathbf{Q}^h, \mathbf{K}^h, \mathbf{V}^h$ 의 shape는 모두 $[B, T, d_h]$ 다.

destination i 와 source j 사이의 attention score는

$$S_{b,i,j}^h = \frac{\langle \mathbf{q}_{b,i}^h, \mathbf{k}_{b,j}^h \rangle}{\sqrt{d_h}} + M_{i,j} \quad (2.10)$$

이다. causal mask는

$$M_{i,j} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i \end{cases} \quad (2.11)$$

로 두며, 실제 유한정밀도 코드에서는 dtype에 맞는 매우 작은 값을 사용할 수도 있다. source 축에 row-wise softmax를 적용하면

$$A_{b,i,j}^h = \frac{\exp S_{b,i,j}^h}{\sum_{k=1}^T \exp S_{b,i,k}^h}, \quad \sum_j A_{b,i,j}^h = 1 \quad (2.12)$$

을 얻는다. masked position의 weight는 이상적인 실수 연산에서 0이다.

한 head의 출력은 source별 value의 attention-weighted sum이다.

$$\mathbf{o}_{b,i}^h = \sum_{j=1}^T A_{b,i,j}^h \mathbf{v}_{b,j}^h. \quad (2.13)$$

A^h 의 shape는 $[B, T, T]$ 이고 O^h 의 shape는 $[B, T, d_h]$ 다. causal mask 때문에 position i 의 출력은 $j \leq i$ 인 value만 직접 사용한다.

제한된 해석 · “정보를 가져온다”는 직관

식 (2.13) 때문에 attention을 source position에서 value 정보를 가져오는 연산이라고 설명할 수 있다. 다만 어떤 source를 선택하는지는 query와 key의 문맥 의존적 계산이며, 전달되는 내용도 원 residual vector 자체가 아니라 value projection이다. attention weight만 보아서는 output projection 이후의 효과를 알 수 없다.

2.5 multi-head attention과 output projection

multi-head attention은 H 개 head의 출력을 이어 붙인 뒤 output projection으로 residual 차원에 되돌린다. head 출력을 마지막 축에서 이어 붙이면

$$\mathbf{O}_{\text{cat}} = \text{Concat}(\mathbf{O}^1, \dots, \mathbf{O}^H) \in R^{B \times T \times H d_h}. \quad (2.14)$$

$H d_h = d$ 인 일반적인 설정에서는 output projection $\mathbf{W}_O \in R^{d \times d}$ 를 곱하여

$$\Delta \mathbf{X}_{\text{attn}} = \mathbf{O}_{\text{cat}} \mathbf{W}_O \in R^{B \times T \times d} \quad (2.15)$$

를 얻는다. 이를 현재 residual stream에 더할 수 있는 shape로 되돌리는 과정이다.

$\mathbf{W}_O$ 를 head별 row block으로 나누면 전체 출력은 head별 residual update의 합으로도 쓸 수 있다.

$$\Delta \mathbf{X}_{\text{attn}} = \sum_{h=1}^H \mathbf{O}^h \mathbf{W}_O^h. \quad (2.16)$$

따라서 concatenation 구현과 head별 합 표현은 서로 다른 모델이 아니라 같은 선형연산의 두 표기다 (Elhage et al., 2021).

attention pattern A^h 는 input에 의존하며 softmax 때문에 비선형이다. 그러나 특정 input에서 A^h 를 고정하면 value와 output projection을 거치는 경로는 residual activation에 대해 선형이다. 회로 분석이 QK와 OV 경로를 분리하는 이유가 여기에 있다.

2.6 MLP update와 잔차 덧셈

각 position에 독립적으로 적용되는 기본 MLP는

$$\mathbf{G} = \phi(\mathbf{R}\mathbf{W}_{\text{in}} + \mathbf{b}_{\text{in}}), \quad (2.17)$$

$$\Delta\mathbf{X}_{\text{mlp}} = \mathbf{G}\mathbf{W}_{\text{out}} + \mathbf{b}_{\text{out}} \quad (2.18)$$

로 쓸 수 있다. 여기서 $\mathbf{W}_{\text{in}} \in R^{d \times d_{\text{ff}}}$, $\mathbf{W}_{\text{out}} \in R^{d_{\text{ff}} \times d}$ 이며 $\mathbf{G}$ 의 shape는 $[B, T, d_{\text{ff}}]$ 다. MLP 자체는 서로 다른 token position을 직접 섞지 않는다. position 사이의 통신은 attention에서 일어난다.

nonlinearity ϕ 가 없다면 두 행렬을 하나로 합칠 수 있다. 비선형함수 또는 gating이 있기 때문에 MLP는 입력에 따라 서로 다른 중간 feature를 활성화하고 단일 선형사상보다 풍부한 함수를 표현한다. 현대 모델의 SwiGLU는 두 input projection을 사용하는 gated 형태이므로 이 기본식과 parameterization이 다르다.

한 sequential pre-norm block의 전체 residual update는

$$\mathbf{U} = \mathbf{X} + \Delta\mathbf{X}_{\text{attn}}, \quad (2.19)$$

$$\mathbf{X}_{\text{next}} = \mathbf{U} + \Delta\mathbf{X}_{\text{mlp}} \quad (2.20)$$

이다. 덧셈으로 인해 이전 residual state로 향하는 identity path가 항상 존재한다. 다만 normalization과 이후 layer가 개별 update를 비선형적으로 다시 읽으므로, 마지막 logit에 대한 각 update의 총 효과를 단순히 독립적인 상수 기여로 볼 수는 없다.

수학적 사실 또는 명시된 구현

“hidden state가 layer마다 완전히 교체된다”는 설명보다 “attention과 MLP가 residual stream에 update를 더한다”는 설명이 residual architecture의 계산을 더 직접적으로 반영한다. 그러나 normalization 위치와 parallel/sequential block 여부는 모델마다 다르므로 실제 코드를 확인해야 한다.

2.7 마지막 정규화, unembedding과 logits

unembedding 또는 LM head는 마지막 normalization 결과를 vocabulary logit으로 보낸다. $\mathbf{H} \in R^{B \times T \times d}$ 라고 두면

$$\mathbf{Z}_{b,t,:} = \mathbf{H}_{b,t,:} \mathbf{W}_U^T + \mathbf{b}_U, \quad \mathbf{W}_U \in R^{V \times d} \quad (2.21)$$

를 계산한다. $\mathbf{Z}$ 의 shape는 $[B, T, V]$ 다. prompt 다음 token을 예측할 때는 마지막 유효 position T 의 logits를 사용한다. 훈련에서는 보통 모든 유효 position의 logits를 동시에 계산하여 각 position의 다음 token loss를 구한다.

token v 의 logit은 row-vector 관례에서

$$z_v = \langle \mathbf{h}_T, \mathbf{w}_{U,v} \rangle + b_{U,v} \quad (2.22)$$

이다. residual update $\Delta\mathbf{h}$ 가 normalization을 거치지 않는 이상적인 선형 경로에서 주는 direct logit contribution은 $\langle \Delta\mathbf{h}, \mathbf{w}_{U,v} \rangle$ 다. 실제 모델의 final normalization을 무시하면 오차가 생길

수 있으므로 direct logit attribution의 정의를 명시해야 한다.

수학적 사실 또는 명시된 구현

logit은 normalization 전의 score이며 확률이 아니다. softmax는 vocabulary의 모든 logit을 함께 사용한다. 특정 token의 logit이 증가해도 다른 token의 logit이 더 많이 증가하면 그 token의 확률은 감소할 수 있다.

2.8 autoregressive generation

한 번의 순전파가 분포 $p(x_{T+1} | x_{1:T})$ 를 만들면 decoder가 token $\hat{x}_{T+1}$ 을 선택한다. 대표적인 규칙은 다음과 같다.

- greedy decoding은 $\arg \max_v z_v$ 를 선택한다.
- temperature sampling은 $\text{softmax}(z/\tau)$ 에서 표본을 뽑는다.
- top- k sampling은 상위 k 개 후보 밖의 질량을 제거하고 다시 정규화한다.
- nucleus 또는 top- p sampling은 누적 확률이 기준 p 에 도달하는 최소 후보 집합을 선택한 뒤 다시 정규화한다 (Holtzman et al., 2020).

선택한 token을 sequence 뒤에 붙여 같은 조건부 예측을 반복한다.

$$p(x_{T+1:T+n} | x_{1:T}) = \prod_{r=1}^n p(x_{T+r} | x_{1:T+r-1}). \quad (2.23)$$

따라서 한 응답은 단 한 번의 “전체 문장 출력”이 아니라 token별 조건부 선택의 연쇄다. greedy에서는 선택이 결정론적이지만, 앞 단계에서 선택한 token이 이후 모든 조건부분포의 입력을 바꾸므로 초기 차이는 계속 전파될 수 있다.

구현에서 “temperature 0”은 대개 0으로 나누어 softmax를 계산한다는 뜻이 아니라 sampling을 끄고 argmax를 선택한다는 설정 이름이다.

2.9 완전한 shape 추적 예제

완전한 shape 추적은 token ID부터 next-token logits까지 각 축의 의미를 보존해야 한다. 표 2.2는 $B = 1, T = 8, d = 4096, H = 32, d_h = 128$ 인 MHA block을 가정하며, vocabulary와 MLP 크기는 예시로 각각 128,256, 14,336을 사용한다.

GQA 모델에서는 key와 value의 head 수가 query head 수보다 작으므로 위 key/value shape가 달라진다. 이 차이는 장 3에서 정확히 전개한다.

2.10 Phase 2 학습 점검

학습 점검

1. 길이 3, head dimension 2인 causal attention의 score, mask, softmax와 value 가중합을 손으로 계산한다.
2. query의 position과 key/value의 position을 문장으로 설명하고 attention matrix의

표 2.2: 한 block과 LM head의 shape 추적

대상	계산	shape
token IDs	tokenizer 출력	[1, 8]
embedding/residual	$E[t]$	[1, 8, 4096]
query	$\text{reshape}(RW_Q)$	[1, 32, 8, 128]
key	$\text{reshape}(RW_K)$	[1, 32, 8, 128]
value	$\text{reshape}(RW_V)$	[1, 32, 8, 128]
scores/weights	QK^T , softmax	[1, 32, 8, 8]
head output	AV	[1, 32, 8, 128]
attention update	concat, W_O	[1, 8, 4096]
MLP intermediate	gated/nonlinear projection	[1, 8, 14336]
MLP update	down projection	[1, 8, 4096]
logits	final norm, LM head	[1, 8, 128256]
next-token logits	마지막 position 선택	[1, 128256]

행과 열을 표시한다.

3. 한 head의 concatenation 표현을 head별 residual update의 합으로 바꿔 쓴다.
4. pre-norm sequential block의 모든 중간 tensor shape를 적는다.
5. 마지막 residual update가 특정 token logit에 미치는 direct contribution을 unembedding vector와의 내적으로 계산한다.
6. greedy, temperature, top- k , top- p 가 모델 순전파와 token 선택 중 어느 부분을 바꾸는지 구분한다.

한계 또는 열린 문제

이 장의 수식은 “어떻게 계산되는가”에 완전한 답을 줄 수 있다. 그러나 특정 head가 왜 Obama와 출생지의 관계를 처리하는지, MLP activation이 어떤 feature를 나타내는지, 어느 경로가 Honolulu logit에 인과적으로 필요했는지는 수식만으로 정해지지 않는다.

제 III 부

Phase 3 · 현대 decoder-only LLM

3 현대 LLM의 구성요소와 실행 경로

3.1 결론부터: 현대 모델은 하나의 고정 설계가 아니다

“현대 LLM architecture”라는 단일 표준은 없다. 공개 모델들은 decoder-only causal Transformer라는 공통 골격을 자주 사용하지만 normalization, position encoding, MLP gating, attention head grouping, bias, weight tying, mixture-of-experts 여부와 context 확장법은 서로 다르다. 따라서 정확한 순전파를 재현하려면 논문만이 아니라 해당 checkpoint의 configuration과 실행 코드를 함께 확인해야 한다.

이 장은 dense Llama 계열에서 널리 알려진 다음 경로를 대표 사례로 사용한다.

$$\text{RMSNorm} \rightarrow \text{GQA with RoPE} \rightarrow + \rightarrow \text{RMSNorm} \rightarrow \text{SwiGLU} \rightarrow +. \quad (3.1)$$

Meta가 공개한 Llama 3 reference implementation은 attention과 feed-forward 전에 RMSNorm을 적용하고, query와 key에 rotary embedding을 적용하며, KV head를 query head 수에 맞게 반복하고, SiLU-gated feed-forward를 계산한다 (Meta AI, 2024; Grattafiori et al., 2024). 이는 특정 공개 구현에 관한 사실이지 모든 LLM에 관한 정의가 아니다.

3.2 decoder-only와 causal attention

decoder-only autoregressive language model은 encoder 없이 causal self-attention block을 반복하여 다음 joint distribution을 모델링한다.

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t | x_{<t}) \quad (3.2)$$

원래 Transformer는 encoder와 decoder를 함께 사용하여 machine translation을 수행했지만 (Vaswani et al., 2017), GPT 계열은 position t 의 activation이 $x_{>t}$ 에 의존하지 않도록 미래 position을 attention에서 가린다.

“decoder-only”는 원래 encoder-decoder Transformer의 decoder를 그대로 떼었다는 뜻으로만 이해하면 부정확할 수 있다. language-model decoder에는 encoder output을 읽는 cross-attention sublayer가 없고, causal self-attention과 position-wise MLP가 주된 block을 이룬다. prompt와 생성된 token도 같은 stream에 놓인다.

causal mask는 정보 의존성을 제한하지만 계산 순서를 항상 token별 직렬로 만들지는 않는다. 훈련과 prompt prefill에서는 전체 sequence의 query를 한 번에 병렬 계산할 수 있다. 생성 단계에서는 아직 존재하지 않는 다음 token 때문에 token별 반복이 필요하다.

3.3 pre-normalization과 RMSNorm

LayerNorm은 feature 축의 평균과 분산을 사용한다.

$$\text{LN}(x) = g \odot \frac{x - \mu(x)}{\sqrt{\sigma^2(x) + \epsilon}} + b. \quad (3.3)$$

RMSNorm은 평균을 빼지 않고 root mean square로 scale을 정규화한다 (Zhang and Sennrich, 2019).

$$\text{RMSNorm}(x) = g \odot \frac{x}{\sqrt{d^{-1} \sum_{i=1}^d x_i^2 + \epsilon}}. \quad (3.4)$$

g 는 학습되는 coordinate-wise gain이다. 구현에 따라 bias를 두지 않는다.

post-norm block은 sublayer와 residual addition 뒤에 normalization을 적용한다. pre-norm block은 sublayer가 residual stream을 읽기 전에 normalization을 적용한다. 두 배치는 함수가 다르며 checkpoint의 가중치를 그대로 교환해 사용할 수 없다. pre-norm의 초기 gradient 거동을 분석한 연구는 post-norm보다 warm-up 의존성이 낮아질 수 있는 조건을 제시했지만, 이것이 모든 학습 설정에서 항상 더 우수하다는 정리는 아니다 (Xiong et al., 2020).

수학적 사실 또는 명시된 구현

RMSNorm은 residual stream 자체를 매번 unit RMS로 덮어쓰지 않는다. pre-norm 모델에서는 정규화된 사본을 attention이나 MLP가 읽고, sublayer output은 정규화되지 않은 residual stream에 더해진다.

3.4 RoPE

Rotary Position Embedding은 짝을 이룬 query와 key 좌표를 position에 따라 회전시킨다 (Su et al., 2024). 한 2차원 coordinate pair에 대해

$$R_{\theta,t} = \begin{bmatrix} \cos(t\theta) & -\sin(t\theta) \\ \sin(t\theta) & \cos(t\theta) \end{bmatrix} \quad (3.5)$$

를 적용한다. 여러 frequency θ_r 에 대한 block-diagonal rotation이 전체 head dimension을 구성한다.

RoPE를 적용한 query-key 내적은 두 position의 상대적 offset을 포함한다. position i, j 의 회전된 query와 key를 $\tilde{q}_i = R_i q_i$, $\tilde{k}_j = R_j k_j$ 라고 하면

$$\tilde{q}_i^\top \tilde{k}_j = q_i^\top R_i^\top R_j k_j = q_i^\top R_{j-i} k_j. \quad (3.6)$$

따라서 attention score에 상대적 offset $j-i$ 가 구조적으로 들어간다. RoPE는 일반적으로 value를 회전하지 않으며, base frequency와 장문 context scaling 방식은 모델별로 확인해야 한다.

한계 또는 열린 문제

RoPE가 훈련 길이 밖의 모든 position에 안정적으로 일반화된다는 보장은 위 대수에서 나오지 않는다. context extension은 별도의 scaling 방법과 평가가 필요한 경험적 문제다.

3.5 SwiGLU

GLU 계열은 두 affine projection 가운데 하나를 gate로 사용한다 (Shazeer, 2020). bias를 생략한 대표적인 SwiGLU feed-forward는

$$\text{SwiGLU}(x) = W_{\text{down}} \left(\text{SiLU}(W_{\text{gate}}x) \odot (W_{\text{up}}x) \right), \quad (3.7)$$

$$\text{SiLU}(a) = a \sigma(a) \quad (3.8)$$

로 쓸 수 있다. 행렬 방향은 column-vector 표기다. 코드가 row-vector를 사용하면 전치된 shape로 나타난다.

SwiGLU에서는 gate projection의 activation이 up projection이 전달하는 내용을 조절한다. $W_{\text{gate}}, W_{\text{up}}$ 은 d 에서 d_{ff} 로, W_{down} 은 다시 d 로 보내며, 두 중간 벡터를 element-wise product로 결합한다.

“SwiGLU가 MLP의 지식을 저장한다”는 문장은 수학적 정의가 아니다. 수학적으로 확정된 내용은 식 (3.7)의 함수와 parameter shape다. feature나 memory에 관한 설명은 가중치와 activation을 분석한 경험적 가설이다.

3.6 MHA, MQA와 GQA

다중 헤드 어텐션(MHA)은 query, key, value에 같은 수의 head를 둔다. query head 수를 H_q , key/value head 수를 H_{kv} 라고 하면 MHA에서는 $H_q = H_{kv}$ 다. 다중 질의 어텐션(MQA)은 여러 query head가 하나의 key head와 value head를 공유하므로 $H_{kv} = 1$ 이다 (Shazeer, 2019). 그룹 질의 어텐션(GQA)은 그 사이에 있으며 $1 < H_{kv} < H_q$ 인 구성을 허용한다 (Ainslie et al., 2023).

GQA는 각 query head를 하나의 shared key/value head에 대응시킨다. H_q 가 H_{kv} 의 배수이고 $g = H_q/H_{kv}$ 일 때, query head h 가 사용하는 key/value head의 번호는 대표적으로

$$r(h) = \left\lfloor \frac{h}{g} \right\rfloor \quad (3.9)$$

로 정한다. 그러면 head h 의 계산은

$$A^h = \text{softmax} \left(\frac{Q^h (K^{r(h)})^T}{\sqrt{d_h}} + M \right), \quad (3.10)$$

$$O^h = A^h V^{r(h)} \quad (3.11)$$

가 된다. “key/value head를 반복한다”는 구현은 같은 $K^{r(h)}$ 와 $V^{r(h)}$ 를 여러 query head에 대응시키기 위한 shape 조작이다. 실제 데이터를 복사하지 않고 broadcast하거나 특화된 kernel에서 직접 공유할 수도 있다.

decode 단계에서 매 token마다 저장하는 key와 value 원소 수는 대략 $2LH_{kv}d_h$ 다. 따라서 H_{kv} 를 줄이면 KV cache의 메모리 대역폭과 저장량을 줄일 수 있다. 그러나 MHA, MQA, GQA의 품질과 속도 차이는 모델 크기, 학습법, kernel과 hardware에 의존한다. GQA 논문은 MHA checkpoint를 적은 추가 학습으로 GQA로 바꾸는 방법과 그 실험 결과를 제시했지만, 모든 모델에서 같은 trade-off를 보장하는 정리는 아니다 (Ainslie et al., 2023).

수학적 사실 또는 명시된 구현

GQA는 query의 수를 줄이는 방법이 아니다. query head는 여러 개를 유지하되, 각 그룹이 같은 key와 value를 읽는다. 따라서 attention weight는 query head마다 다를 수 있다.

3.7 tokenization과 vocabulary

모델은 문자열을 직접 받지 않는다. tokenizer가 byte 또는 Unicode 문자열을 vocabulary의 정수 ID 열로 바꾼다. Byte Pair Encoding(BPE)은 반복적으로 빈도가 높은 인접 symbol pair를 합치는 압축 알고리즘에서 출발했으며, 신경망 번역에서는 subword vocabulary를 만드는 방법으로 사용되었다 (Sennrich et al., 2016). SentencePiece는 사전 분절된 단어를 요구하지 않고 raw sentence에서 subword model을 학습하는 구현을 제안했다 (Kudo and Richardson, 2018).

token과 단어는 일반적으로 일대일로 대응하지 않는다. tokenization 함수 τ 와 detokenization 함수 δ 를

$$\tau : S \rightarrow \{0, \dots, V-1\}^*, \quad \delta : \{0, \dots, V-1\}^* \rightarrow S \quad (3.12)$$

로 쓰자. 여기서 S 는 문자열 집합이고 별표는 길이가 가변적인 열을 뜻한다. 공백, 구두점, byte 조각 또는 여러 문자가 한 token이 될 수 있다. 같은 표면 문자열도 앞 공백과 normalization 방식에 따라 서로 다른 ID 열이 될 수 있다.

tokenizer는 모델 가중치와 분리된 임의의 전처리가 아니다. embedding과 LM head의 row 의미가 vocabulary ID에 고정되어 있으므로 checkpoint와 맞는 tokenizer를 사용해야 한다. 특수 token의 ID, 문장 시작과 끝 처리, 대화 template도 실제 입력 열을 바꾼다.

한계 또는 열린 문제

모델이 “몇 단어를 보았다”는 표현은 token 수를 정확히 나타내지 않는다. 계산량, context 길이와 loss를 재현하려면 원 문자열뿐 아니라 tokenizer 버전, normalization, special-token 설정과 최종 token IDs를 기록해야 한다.

3.8 KV cache, prefill과 decode

길이 T 의 prompt를 처음 처리하는 단계를 prefill이라고 한다. 모든 prompt position의 query, key와 value를 만들고 causal attention을 병렬로 계산한 뒤, 각 layer의 key와 value를 cache에 보관할 수 있다. 이어서 token 한 개를 생성하는 decode 단계에서는 새 position의 query, key와 value만 계산하고, 새 query가 과거의 cached key/value와 새 key/value를 함께 읽는다.

decode의 새 query는 과거 cache에 새 key와 value를 붙인 전체 prefix를 읽는다. layer ℓ 의 과거 cache를 $\mathbf{K}_{1:T}^{(\ell)}, \mathbf{V}_{1:T}^{(\ell)}$ 라고 하고, 새 token의 projection을 $\mathbf{q}_{T+1}, \mathbf{k}_{T+1}, \mathbf{v}_{T+1}$ 로 쓰면

$$\mathbf{K}_{1:T+1} = \text{Append}(\mathbf{K}_{1:T}, \mathbf{k}_{T+1}), \quad (3.13)$$

$$\mathbf{V}_{1:T+1} = \text{Append}(\mathbf{V}_{1:T}, \mathbf{v}_{T+1}), \quad (3.14)$$

$$\mathbf{o}_{T+1} = \text{softmax}\left(\frac{\mathbf{q}_{T+1} \mathbf{K}_{1:T+1}^\top}{\sqrt{d_h}}\right) \mathbf{V}_{1:T+1} \quad (3.15)$$

를 계산한다. 과거 token의 key와 value는 다시 계산하지 않는다.

cache가 없으면 생성 step s 마다 앞선 $T + s$ 개 position의 K/V를 다시 만들어야 한다. cache는 이 중복 projection을 없애지만, sequence 길이에 비례하는 저장 공간과 매 step의 cache 읽기 비용이 필요하다. batch의 sequence가 서로 다른 속도로 끝나면 memory block을 효율적으로 배치하는 별도 시스템 문제가 생긴다. PagedAttention은 운영체제의 가상 메모리와 비슷한 block 단위 관리로 KV cache의 낭비와 공유 문제를 다루는 시스템을 제안했다 (Kwon et al., 2023).

표 3.1: prefill과 한 token decode의 계산 경로

구분	prefill	decode 한 단계
입력 position	prompt 전체	새 position 한 개
새로 만드는 Q/K/V	prompt 전체	새 position만
attention이 읽는 K/V	각 position의 허용된 prefix	과거 cache와 새 position
주된 병렬 축	많은 token position	batch, head 및 내부 행렬 연산
주된 자원 병목	큰 행렬 연산과 메모리	cache 대역폭과 작은 연산의 반복
출력 사용	보통 마지막 prompt logit	새 token의 logit

KV cache는 수학적 모델의 조건부분포를 바꾸지 않는 재사용 장치다. 다만 dtype, kernel, 양자화와 연산 순서가 바뀌면 유한정밀도 결과가 완전히 bit-wise 동일하지 않을 수 있다.

3.9 FlashAttention과 수치적 동등성

표준 attention을 순진하게 구현하면 $S = QK^T$ 와 $A = \text{softmax}(S)$ 를 고대역폭 메모리에 저장하므로 sequence 길이에 대해 quadratic한 중간 메모리가 든다. FlashAttention은 attention 행렬을 block으로 나누고 on-chip SRAM과 고대역폭 메모리 사이의 읽기와 쓰기를 줄이는 순서를 사용한다. 원 논문은 근사 attention이 아니라 exact attention algorithm으로 제시하며, 역전파에 필요한 중간값도 재계산과 online softmax로 관리한다 (Dao et al., 2022).

online softmax는 block별 최대값과 지수합을 재조정하여 전체 row의 softmax를 보존한다. 지금까지 처리한 score의 최대값과 지수합을 m, l 이라고 하고 새 block의 최대값과 지수합을 m_b, l_b 라고 할 때, 병합식은

$$m' = \max(m, m_b), \quad (3.16)$$

$$l' = e^{m-m'}l + e^{m_b-m'}l_b \quad (3.17)$$

가 된다. value의 가중합도 같은 scale factor로 갱신하면 전체 score를 한꺼번에 materialize하지 않고 같은 실수식의 결과를 얻을 수 있다.

수학적 사실 또는 명시된 구현

FlashAttention은 attention의 모델 구조를 새로운 의미론으로 바꾸는 구성요소가 아니다. 같은 Q, K, V, mask와 softmax attention을 I/O 효율적인 순서로 계산하는 algorithm이다. 따라서 회로를 해석할 때는 수학적 attention과 kernel 구현을 구분해야 한다.

“exact”는 이상적인 연산식에 관한 표현이다. floating-point 덧셈은 결합법칙을 만족하지 않으므로 연산 순서가 바뀌면 마지막 bit가 달라질 수 있다. dropout, mixed precision, kernel

버전과 hardware까지 고정하지 않으면 bit-wise 재현성을 기대해서는 안 된다.

3.10 weight tying과 LM head

weight tying은 입력 embedding과 출력 LM head가 같은 parameter를 공유하는 설계다. 두 행렬 $E, W_U \in R^{V \times d}$ 에 대해

$$W_U = E \quad (3.18)$$

이 등식은 두 모듈이 같은 parameter 저장소를 사용한다는 뜻이다 (Press and Wolf, 2017). untied 모델은 두 행렬을 독립적으로 학습한다. tying 여부는 architecture에 따라 다르며, 같은 shape라는 사실만으로 공유를 결론낼 수 없다.

tied 모델에서도 input embedding으로서의 역할과 unembedding으로서의 역할은 계산 맥이 다르다. 입력에서는 token ID로 row를 선택한다. 출력에서는 마지막 hidden vector와 모든 row의 내적을 한꺼번에 계산한다. final normalization이나 별도 output bias가 있으면 그 연산도 포함해야 한다.

3.11 훈련이 가중치를 만드는 과정

순전과 수식은 가중치가 어떻게 생겼는지 설명하지 않는다. autoregressive pretraining은 corpus의 token 열에서 다음 token 음의 로그가능도를 최소화한다.

$$L_{\text{NTP}}(\theta) = - \sum_{t=1}^{T-1} \log p_{\theta}(x_{t+1} | x_{\leq t}). \quad (3.19)$$

teacher forcing에서는 실제 prefix $x_{\leq t}$ 를 모든 position에 제공하고 loss를 병렬 계산한다. 역전파가 $\nabla_{\theta} L$ 을 구하면 optimizer가 parameter를 갱신한다. Adam은 first moment와 second raw moment의 지수이동평균을 사용하며 (Kingma and Ba, 2015), AdamW는 weight decay를 gradient update에서 분리한다 (Loshchilov and Hutter, 2019).

학습 결과는 architecture만으로 정해지지 않는다. 데이터의 분포와 순서, objective, 초기값, optimizer, learning-rate schedule, batch 구성, 정밀도와 분산 학습의 수치적 세부사항이 최종 parameter에 영향을 준다. 모델 크기, 데이터와 compute 사이의 경험적 scaling 관계도 보고되었지만 (Kaplan et al., 2020; Hoffmann et al., 2022), 이는 특정 실험 범위에 맞춘 경험 법칙이며 개별 능력이나 내부 회로를 직접 결정하는 공식이 아니다.

instruction tuning과 preference optimization은 pretraining 뒤의 출력 분포를 바꾼다. InstructGPT는 supervised demonstrations와 사람 선호로 학습한 reward model, PPO 기반 강화학습을 조합했다 (Ouyang et al., 2022; Schulman et al., 2017). DPO는 명시적인 reward model과 online reinforcement-learning loop 없이 preference pair에 대한 분류형 목적함수를 최적화하는 방법을 유도했다 (Rafailov et al., 2023). 이들은 가능한 post-training 절차의 예이며 모든 공개 또는 상용 LLM의 동일한 recipe가 아니다.

한계 또는 열린 문제

next-token objective를 안다고 해서 학습된 내부 알고리즘을 역으로 유일하게 복원할 수 있는 것은 아니다. 서로 다른 parameterization과 내부 표현이 같은 관측 출력 또는 매우 비슷한 loss를 만들 수 있다. 이 비식별성 때문에 내부 메커니즘에는 별도의 관찰과 개입이 필요하다.

3.12 Phase 3 학습 점검**학습 점검**

1. 공개 checkpoint 하나를 선택하여 layer 수, d , d_{ff} , H_q , H_{kv} , RoPE 설정, normalization의 ϵ 와 weight tying 여부를 표로 만든다.
2. $H_q = 32$, $H_{kv} = 8$, $d_h = 128$ 일 때 각 query head가 공유하는 KV head를 모두 적고, token 한 개당 layer별 cache 원소 수를 계산한다.
3. 같은 문자열을 앞 공백 유무와 Unicode normalization을 바꾸어 token ID로 변환하고 차이를 기록한다.
4. 길이 T 인 prompt의 prefill과 n 개 token decode에서 어떤 K/V projection이 재사용되는지 의사코드로 쓴다.
5. FlashAttention이 줄이는 것은 수학적 연산, HBM I/O, 중간 메모리 중 무엇인지 구분한다.
6. pretraining objective, optimizer와 post-training objective가 각각 순전과 함수의 어떤 parameter를 바꾸는지 설명한다.

수학적 사실 또는 명시된 구현

Phase 3을 마치면 공개 구현의 순전과 코드를 수식과 tensor shape로 대조할 수 있어야 한다. 그러나 내부 벡터의 의미를 읽거나 특정 행동의 원인을 식별하는 일은 아직 시작하지 않았다.

제 IV 부

Phase 4 · 표현을 관찰하는 방법

4 내부 표현을 읽는 방법과 그 한계

4.1 결론부터: 읽을 수 있다는 사실은 사용된다는 뜻이 아니다

활성값에서 어떤 변수를 예측할 수 있다는 결과는 그 변수에 관한 정보가 활성값에 존재한다는 증거다. 그러나 원 모델이 같은 판별 경계나 같은 방향을 실제 출력 계산에 사용한다는 결론은 별도의 인과적 증거가 없으면 성립하지 않는다. 이 장의 핵심 구분은 다음 세 문장이다.

1. 변수 y 를 활성값 h 에서 복원할 수 있다.
2. y 에 대응한다고 해석한 방향을 바꾸면 모델 행동도 바뀐다.
3. 원 모델이 자연스러운 입력에서 그 방향을 이용하는 메커니즘을 안다.

첫 문장은 관찰적이고, 둘째 문장은 개입적이며, 셋째 문장은 구성요소와 경로를 포함한 설명이다. 앞 문장이 참이라고 해서 다음 문장이 자동으로 참이 되지는 않는다.

4.2 분산 표현과 특징 방향

분산 표현은 의미 변수가 좌표 하나가 아니라 여러 좌표의 결합과 관계를 맺을 수 있다는 관점이다. layer ℓ , position t 의 residual state를 $h_t^{(\ell)} \in R^d$ 라고 할 때, “개체”, “시제”, “진실성” 같은 변수가 좌표 한 개에만 저장되어야 할 이유는 없다. 한 좌표도 여러 입력 조건에서 서로 다른 변수와 상관될 수 있다.

선형 readout은

$$s(h) = w^T h + b \quad (4.1)$$

로 score를 만든다. binary classification에서는 $s > 0$ 인지로 class를 나눌 수 있다. w 를 feature direction이라고 부를 때는 다음 차이를 유지해야 한다.

- 좌표 또는 neuron은 선택한 basis의 축 하나다.
- 방향은 여러 좌표의 선형결합일 수 있다.
- subspace는 한 개 이상의 독립 방향이 만드는 부분공간이다.
- feature는 연구자가 정한 의미 변수 또는 학습된 dictionary element를 가리킬 수 있으므로, 논문마다 조작적 정의를 확인해야 한다.

개별 neuron의 의미 해석은 모델이 사용하는 basis에 의존한다. invertible matrix A 로 좌표를 $h' = Ah$ 로 바꾸고 다음 layer의 weight를 그에 맞게 보정하면 같은 함수를 표현할 수 있지만 개별 좌표의 값은 달라진다. 따라서 “neuron 1723이 개념 그 자체다”라는 주장은 basis와 입력 분포를 명시하지 않으면 강한 주장이다. 반면 특정 모델의 고정된 basis에서 그 neuron의 활성 조건을 기술하는 것은 검증 가능한 경험적 주장이다.

제한된 해석 · 분산 표현의 최소 의미

분산 표현은 모든 정보가 모든 좌표에 균등하게 흩어져 있다는 뜻이 아니다. 어떤 변수는 소수의 방향이나 neuron에서 더 쉽게 복원될 수 있다. 이 용어는 의미 변수와 parameter 좌표 사이에 필연적인 일대일 대응이 없다는 점을 가리킨다.

4.3 표현 공간의 기하

표현 기하 연구는 활성값 사이의 거리, 각도, 군집, 저차원 구조와 변환 관계를 분석한다. 두 벡터의 cosine similarity는

$$\cos(x, y) = \frac{x^T y}{\|x\|_2 \|y\|_2} \quad (4.2)$$

다. 이는 방향의 일치도를 재지만 norm 차이를 버린다. residual norm이 logit이나 다음 normalization에 영향을 주는 모델에서는 cosine만으로 계산 효과를 판단할 수 없다.

주성분분석은 표본 분산이 큰 방향을 찾지만 그 방향을 인간 개념으로 보장하지 않는다. 데이터 행렬 $H \in R^{n \times d}$ 를 평균 중심화하고 SVD를 적용하면 이 방향을 계산할 수 있다. label에 따라 평균 차이

$$v = E[h | y = 1] - E[h | y = 0] \quad (4.3)$$

를 계산하는 방법도 있다. 이 방향은 표본 분포, layer, position과 prompt 형식에 의존한다.

일부 연구는 개념이나 관계가 선형 구조로 나타난다는 “선형 표현 가설”을 이론과 실험으로 분석한다 (Park et al., 2024). 이 문헌은 특정 정의와 모델에서 선형 구조를 지지하는 결과를 제공하지만, 모든 의미 변수가 한 개의 전역 선형 방향으로 표현된다는 정리를 제공하지 않는다.

한계 또는 열린 문제

군집 그림은 차원 축소 방법과 hyperparameter에 민감할 수 있다. t-SNE나 UMAP의 2차원 배치에서 떨어져 보이는 두 군집이 원래 공간에서도 같은 방식으로 분리된다고 단정해서는 안 된다. 원 차원의 거리, held-out 분류와 개입 결과를 함께 확인해야 한다.

4.4 선형 probe와 control task

probe는 고정된 모델의 활성값을 입력으로 받아 label을 예측하도록 별도 모델을 학습하는 방법이다. 선형 probe의 대표 목적함수는

$$\hat{w}, \hat{b} = \arg \min_{w, b} \sum_{i=1}^n \text{CE}(y_i, \sigma(w^T h_i + b)) + \lambda \|w\|_2^2 \quad (4.4)$$

다. 평가에는 probe를 학습하지 않은 held-out sample을 사용해야 한다.

높은 정확도는 다음 대안들 가운데 무엇이 맞는지 혼자 결정하지 못한다.

- 원 모델이 행동을 만들 때 실제로 사용하는 변수가 복원되었다.
- 입력의 표면적 confound가 label과 함께 복원되었다.
- 고차원 표현에서 probe가 우연한 label mapping을 암기했다.
- 해당 정보는 존재하지만 downstream 계산이 무시한다.

Hewitt와 Liang은 언어 구조 probe가 representation의 구조와 probe 자체의 암기 능력을 혼동할 수 있음을 지적하고, 무작위 control label에 대한 성능과 실제 과제 성능의 차이를 selectivity로 평가했다 (Hewitt and Liang, 2019). 실제 연구에서는 train/test 분리뿐 아니라 lexical overlap, prompt template, token position, class balance와 probe capacity를 통제해야 한다.

probe 결과를 기록하는 양식

재현 가능한 probe 결과에는 모델부터 결론의 증거 수준까지 전체 분석 명세가 필요하다. 모델과 checkpoint, layer와 hook 위치, 사용한 token position, label의 정의, train/validation/test 분리 단위, probe class와 regularization, baseline, control task, metric과 confidence interval을 기록한다. 마지막으로 “정보 존재”와 “인과적 사용” 중 어디까지 결론낼 수 있는지 한 문장으로 제한한다.

4.5 logit lens와 tuned lens

logit lens는 중간 residual state를 최종 vocabulary 공간으로 투영하여 각 layer에서 어떤 token이 선호되는지 관찰한다. 가장 단순한 형태는

$$z_t^{(\ell)} = W_U \text{Norm}_f(h_t^{(\ell)}) + b_U \quad (4.5)$$

다. 최종 layer에 맞춰 학습된 normalization과 unembedding을 중간 layer에 적용하는 진단 도구다. 원 모델이 실제로 중간마다 LM head를 실행한다는 뜻은 아니다.

중간 residual distribution은 최종 residual distribution과 다를 수 있다. tuned lens는 layer별 affine translator A_ℓ, c_ℓ 를 학습하여

$$z_t^{(\ell)} = W_U \text{Norm}_f(A_\ell h_t^{(\ell)} + c_\ell) + b_U \quad (4.6)$$

로 예측을 읽는다 (Belrose et al., 2023). 이는 중간 예측을 더 잘 맞추기 위한 별도 모델이며, translator의 계산이 원 Transformer 안에 존재한다는 증거는 아니다.

logit lens에서 정답 token의 rank가 layer에 따라 오르는 현상은 유용한 관찰이다. 그러나 그 token이 실제 생성에 필요했던 경로를 확인하려면 activation patching이나 path-level 개입 같은 추가 실험이 필요하다.

4.6 중첩과 다의적 neuron

중첩(superposition)은 서로 직교하지 않는 여러 feature 방향을 같은 표현 공간에 배치하는 가설이다. 차원 d 보다 많은 잠재 feature를 표현해야 하고 feature가 희소하게 나타날 때 이런 배치가 가능하다. toy model 연구는 sparsity와 feature importance가 달라질 때 비직교 feature 표현과 다의적 neuron이 나타나는 조건을 구성적으로 보였다 (Elhage et al., 2022).

toy model에서 중첩을 구성한 결과와 자연 LLM에서 중첩을 식별한 결과는 서로 다른 명제다.

1. 설계된 toy model에서 최적화가 중첩 해를 만든다.
2. 자연어로 학습한 대형 LLM의 특정 activation이 같은 이유로 다의적이다.

첫째는 통제된 모델에서 직접 조사할 수 있다. 둘째는 자연 모델의 feature 정의와 측정법을 추가로 요구한다. 한 neuron이 서로 무관해 보이는 여러 문맥에서 활성화된다는 관찰은 다의성의 증거가 될 수 있지만, 그 원인이 오직 중첩이라고 유일하게 식별하지는 못한다.

4.7 희소 오토인코더

희소 오토인코더(SAE)는 activation $\mathbf{x} \in R^d$ 를 더 큰 feature 공간의 희소한 code $\mathbf{f} \in R^m$ 으로 바꾸고 다시 복원한다. 단순한 한 형태는

$$\mathbf{f} = \text{ReLU}(\mathbf{W}_{\text{enc}}(\mathbf{x} - \mathbf{b}_{\text{dec}}) + \mathbf{b}_{\text{enc}}), \quad (4.7)$$

$$\hat{\mathbf{x}} = \mathbf{W}_{\text{dec}}\mathbf{f} + \mathbf{b}_{\text{dec}}, \quad (4.8)$$

$$L_{\text{SAE}} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + \lambda \|\mathbf{f}\|_1 \quad (4.9)$$

로 쓸 수 있다. $m > d$ 인 overcomplete dictionary를 사용할 수 있다. decoder의 column은 activation 공간에 쓰는 방향이고, code coordinate는 해당 방향의 활성 세기다.

희소 dictionary가 사람이 해석하기 쉬운 feature를 줄 수 있다는 대규모 사례 연구가 보고되었다 (Bricken et al., 2023; Templeton et al., 2024). 그러나 reconstruction loss와 sparsity penalty는 의미론적 유일성을 직접 최적화하지 않는다. dictionary는 random seed, 데이터, sparsity 강도와 architecture에 따라 달라질 수 있으며, reconstruction error가 원 모델 계산의 일부를 잃는다.

SAEBench는 여러 SAE architecture를 여덟 종류의 metric으로 비교했고, 일반적인 proxy metric의 향상이 feature disentanglement나 practical downstream 성능의 향상으로 안정적으로 이어지지 않는 결과를 보고했다 (Karvonen et al., 2025). 따라서 낮은 reconstruction error 또는 높은 sparsity 하나만으로 “해석가능성이 좋아졌다”고 결론내리면 안 된다.

한계 또는 열린 문제

SAE feature에 자연어 이름을 붙이는 일은 가설 생성이다. feature가 활성화되는 positive와 negative example, 유사 feature, activation distribution, decoder 방향 개입, downstream 효과와 reconstruction error 경로를 함께 조사해야 한다. 자동 설명 모델의 문장이 그 feature의 완전한 정의가 되는 것은 아니다.

4.8 표현 분석의 최소 증거 기준

표현 분석의 산출물은 결론문보다 재현 가능한 object여야 한다. activation을 뽑은 정확한 hook, tensor shape, 표본 ID, 전처리, probe weight, metric과 seed를 보존해야 다음 장의 회로 분석과 인과 개입으로 이어갈 수 있다.

표 4.1: 표현에 관한 주장과 필요한 추가 증거

주장	최소 관찰	아직 남는 질문
상관이 있다	독립 표본에서 activation과 label의 관계가 재현된다.	confound인가?
선형 복원이 된다	held-out probe가 적절한 baseline과 control을 넘는다.	원 모델이 사용하는가?
방향이 행동을 제어한다	크기와 부작용을 기록한 intervention이 행동을 바꾼다.	자연 입력에서 같은 경로를 쓰는가?
구성요소가 필요하다	적절한 ablation에서 target metric이 나빠진다.	중복 경로나 self-repair가 있는가?
메커니즘을 설명한다	경로 가설이 여러 반사실 입력과 개입 결과를 예측한다.	분포와 모델을 넘어 일반화하는가?

4.9 Phase 4 학습 점검

학습 점검

1. 같은 의미 label에 대해 neuron 하나, mean-difference direction과 regularized linear probe를 비교하고 held-out 성능을 기록한다.
2. label을 무작위로 섞은 control task와 lexical split을 추가하여 selectivity를 평가한다.
3. 각 layer의 logit lens와 tuned lens 분포를 비교하고, 어느 계산이 원 모델 밖에서 학습되었는지 표시한다.
4. SAE 하나를 학습하여 reconstruction error, 평균 활성 feature 수, dead feature 비율과 intervention 결과를 함께 기록한다.
5. 모든 그림과 표의 결론을 “복원”, “개입”, “메커니즘” 가운데 하나로 분류한다.

수학적 사실 또는 명시된 구현

Phase 4의 종료 조건은 activation에 이름을 많이 붙이는 것이 아니다. 무엇이 관찰되었고, 어떤 대안 설명이 남으며, 다음 인과 실험이 무엇인지를 정확히 말할 수 있어야 한다.

제 V 부

Phase 5 · 회로와 내부 알고리즘

5 잔차 스트림, 회로와 내부 알고리즘

5.1 결론부터: 회로는 행동에 상대적인 계산 부분그래프다

회로는 명시된 입력 분포와 행동 metric에 대해 모델의 계산을 충분히 재현하거나 설명한다고 가정한 구성요소와 연결의 부분그래프다. Transformer 전체도 하나의 계산그래프지만, 모든 edge를 회로라고 부르면 설명 단위가 너무 커진다. 따라서 “모델의 회로”라는 표현보다 다음 tuple이 정확하다.

$$C = (\text{model}, \text{input distribution}, \text{behavior}, \text{nodes}, \text{edges}, \text{intervention rule}). \quad (5.1)$$

같은 모델도 이름 복사, 사실 회상과 거절에는 다른 회로 가설을 가질 수 있다. 같은 행동도 prompt 형식이나 metric이 바뀌면 중요한 경로가 달라질 수 있다. 회로가 발견되었다는 말은 이 tuple의 범위에서 평가해야 한다.

한계 또는 열린 문제

“회로”에는 분야 전체가 합의한 유일한 원자 단위가 없다. node를 attention head, MLP block, neuron, SAE feature 또는 token-position별 activation으로 잡을 수 있다. 서로 다른 분해의 회로 크기와 완전성을 그대로 비교해서는 안 된다.

5.2 residual stream을 통신 채널로 보기

pre-norm Transformer에서 각 sublayer는 residual stream의 정규화된 사본을 읽고 d 차원의 update를 다시 쓴다. 단순화하면 마지막 residual state는

$$\mathbf{x}_t^{(L)} = \mathbf{x}_t^{(0)} + \sum_{\ell=0}^{L-1} \Delta \mathbf{x}_{\text{attn},t}^{(\ell)} + \sum_{\ell=0}^{L-1} \Delta \mathbf{x}_{\text{mlp},t}^{(\ell)} \quad (5.2)$$

로 정확히 쓸 수 있다. 각 update의 값은 앞선 모든 update를 normalization과 비선형 연산으로 읽은 결과이므로 서로 독립적인 항은 아니다.

Elhage 등은 이 구조를 여러 구성요소가 공용 residual stream을 통해 읽고 쓰는 통신 채널로 해석하고, attention-only 소형 모델에서 경로 전개와 QK/OV 분해를 체계화했다 (Elhage et al., 2021). 이 관점은 architecture의 덧셈 구조와 잘 맞지만, residual vector의 의미가 자연스럽게 분리된 slot으로 보장된다는 뜻은 아니다. 서로 다른 update가 같은 subspace를 사용하거나 상쇄할 수 있다.

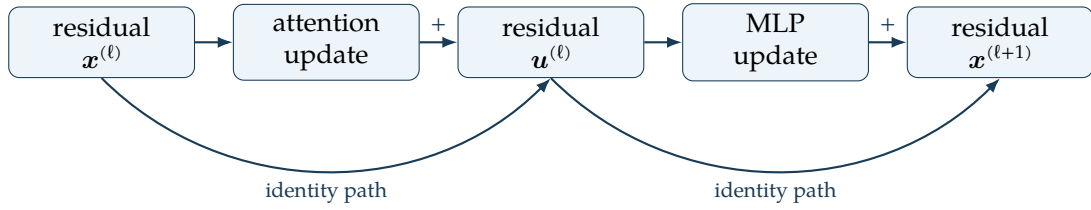


그림 5.1: 한 block에서 residual stream을 읽고 update를 더하는 경로

5.3 direct logit attribution과 경로 전개

DLA는 final normalization을 선형화하거나 생략한 조건에서 residual update와 unembedding의 내적으로 직접 기여를 배정한다. 이 조건에서 token v 의 logit은

$$z_v = \mathbf{w}_{U,v}^\top \mathbf{x}_t^{(L)} + b_v \quad (5.3)$$

다. 식 (5.2)을 대입하면 embedding과 각 sublayer update가 주는 직접 내적을 계산할 수 있다. 이를 direct logit attribution(DLA)이라고 부른다. 비교 대상 token v^+ 와 v^- 의 logit difference를 쓰면 구성요소 c 의 직접 기여는

$$\text{DLA}_c = (\mathbf{w}_{U,v^+} - \mathbf{w}_{U,v^-})^\top \Delta \mathbf{x}_c \quad (5.4)$$

로 둘 수 있다.

DLA는 residual에 최종적으로 쓰인 방향이 unembedding과 얼마나 정렬되는지 측정한다. 구성요소 c 가 이후 layer의 query, key 또는 MLP gate를 바꾸는 간접 효과는 식 (5.4)에 포함되지 않는다. final LayerNorm이나 RMSNorm도 모든 성분을 결합하므로 정확한 attribution 규칙을 명시해야 한다.

경로 전개는 residual connection을 따라 계산을 합성하여 입력에서 logit까지의 경로를 열거하는 대수적 관점이다. attention pattern과 normalization scale을 특정 입력에서 고정하면 attention의 value-output 경로는 선형이므로 matrix product로 합칠 수 있다. 그러나 MLP activation, attention softmax와 normalization scale은 원래 입력 의존적이다. 이를 고정된 전개는 해당 입력 주변의 조건부 설명이지 전역 선형 모델이 아니다.

5.4 QK 회로와 OV 회로

QK 회로는 destination 표현과 source 표현 사이의 attention score를 정하는 bilinear form이다. column-vector convention으로 정규화된 residual을 $\mathbf{r}_i$ 라고 두고

$$\mathbf{q}_i = \mathbf{W}_Q^\top \mathbf{r}_i, \quad \mathbf{k}_j = \mathbf{W}_K^\top \mathbf{r}_j, \quad \mathbf{v}_j = \mathbf{W}_V^\top \mathbf{r}_j \quad (5.5)$$

라고 하자. score의 핵심 bilinear form은

$$\mathbf{r}_i^\top \underbrace{\mathbf{W}_Q \mathbf{W}_K^\top}_{\text{QK matrix}} \mathbf{r}_j \quad (5.6)$$

다. 이 form은 어떤 destination 표현이 어떤 source 표현을 선택하는지 결정한다.

OV map은 선택된 source의 value가 residual stream에 쓰는 내용을 정한다. 그 update는

$$\Delta \mathbf{x}_i = \sum_j A_{ij} \underbrace{\mathbf{W}_O^T \mathbf{W}_V^T}_{\text{OV map}} \mathbf{r}_j \quad (5.7)$$

로 나타낼 수 있다. 정확한 전치 방향은 코드의 row/column convention에 따라 달라지지만, 선택을 정하는 QK와 전달 내용을 정하는 OV를 분리한다는 요점은 같다 (Elhage et al., 2021).

큰 attention weight는 QK 선택에 관한 관찰이다. OV map이 source 정보를 target logit과 반대되는 방향으로 쓰거나 거의 0으로 보내면 출력 기여는 작거나 음수일 수 있다. 반대로 작은 weight도 매우 큰 value-output vector와 결합되면 무시할 수 없을 수 있다.

5.5 head composition

앞 layer의 head가 residual에 쓴 vector는 뒤 head의 query, key와 value projection이 모두 읽을 수 있다. 이에 따라 세 가지 대표 합성이 생긴다.

Q-composition

앞 head의 출력이 뒤 head의 query를 바꾸어 destination이 찾는 source를 조절한다.

K-composition

앞 head의 출력이 뒤 head의 key를 바꾸어 특정 source가 선택될 가능성을 조절한다.

V-composition

앞 head가 한 position에 쓴 내용을 뒤 head가 value 경로로 다른 position에 운반한다.

이 분류는 attention-only 모델의 virtual weight를 분석하는 데 유용하다. 실제 깊은 모델에서는 normalization, MLP update, 여러 head의 합과 softmax가 사이에 있으므로 matrix product의 크기만으로 인과 효과를 확정할 수 없다. composition score는 후보 경로를 찾는 도구이며 개입 검증이 뒤따라야 한다.

5.6 induction head

induction head가 구현하는 대표 패턴은

$$[A][B] \cdots [A] \rightarrow [B] \quad (5.8)$$

다. 앞에서 A 다음에 B가 나타났다면, 뒤의 A 위치에서 B를 다음 token으로 복사하는 경향이다. 두 layer attention-only 모델의 이상화된 회로에서는 다음과 같이 설명할 수 있다.

1. 앞 layer의 previous-token head가 각 position에 바로 앞 token의 정보를 쓴다.
2. 뒤 layer induction head의 query가 현재 token A를 나타내고, key가 “바로 앞 token이 A였던 source”와 맞는다.
3. 그 source의 value에는 A 다음 token인 B의 정보가 있으므로 OV 경로가 B logit을 높이는 방향으로 이를 복사한다.

Olsson 등은 소형 attention-only 모델에서 induction head와 in-context loss 개선 사이에 강한 인과 증거를 제시하고, MLP가 있는 더 큰 모델에서는 여러 상관적 증거를 보고했다 (Olsson

et al., 2022). 원 논문 자체가 증거 수준을 구분한다. 따라서 “모든 대형 LLM의 in-context learning은 induction head 하나로 완전히 설명된다”고 요약하면 원 결과의 범위를 넘는다.

실험 결과 · Olsson et al. (2022)의 제한된 결과

반복 token pattern, training 중 induction-like head의 출현 시점과 in-context learning metric을 연결한 결과다. 소형 attention-only 모델에 대한 강한 인과 증거와 더 큰 모델에 대한 간접 증거를 같은 수준으로 취급하지 않는다.

5.7 IOI 회로

간접 목적어 식별(IOI)은 “When Mary and John went to the store, John gave a drink to”와 같은 문맥에서 반복되지 않은 이름인 Mary를 예측하는 과제다. Wang 등은 GPT-2 small에서 이 행동을 분석하여 26개 attention head를 7개 주요 기능군으로 묶은 회로를 제시했다. 연구는 name mover, negative name mover, duplicate-token, previous-token, induction 및 S-inhibition 기능을 포함하고, causal intervention과 faithfulness, completeness, minimality 기준으로 설명을 평가했다 (Wang et al., 2022).

이 연구의 중요성은 자연어 행동 하나를 입력에서 logit까지 비교적 세밀한 head-level 경로로 연결했다는 데 있다. 동시에 다음 범위가 명시되어야 한다.

- 분석 대상은 GPT-2 small과 연구가 구성한 IOI distribution이다.
- node granularity는 주로 attention head이며 MLP와 더 미세한 feature를 완전히 의미론적으로 분해하지 않는다.
- 정량 기준이 설명을 지지했지만 저자들은 남은 설명 격차도 보고했다.
- 다른 이름 분포, 문장 구조, 모델과 과제에 동일한 회로가 자동으로 적용되지 않는다.

따라서 IOI는 “자연 모델도 부분적으로 reverse engineering할 수 있다”는 강한 사례지만, “자연어 모델 전체의 알고리즘을 발견했다”는 사례는 아니다.

5.8 MLP feature와 memory 해석

기본 FFN은 중간 neuron별 activation과 residual 출력 방향의 합으로 전개할 수 있다.

$$\text{FFN}(\mathbf{x}) = \sum_{i=1}^{d_{\text{ff}}} \phi(\mathbf{k}_i^T \mathbf{x} + b_i) \mathbf{v}_i + \mathbf{b}_{\text{out}} \quad (5.9)$$

input matrix의 row $\mathbf{k}_i$ 는 activation 조건을 정하고, output matrix의 대응 column $\mathbf{v}_i$ 는 residual에 쓰는 방향을 정한다. 이 대수 때문에 한 neuron을 key-value pair처럼 분석할 수 있다.

Geva 등은 Transformer FFN의 input pattern과 output vocabulary 분포를 조사하고, lower layer에는 상대적으로 얇은 pattern이, upper layer에는 더 semantic한 pattern이 나타나는 경향을 보고하며 “key-value memory” 해석을 제안했다 (Geva et al., 2021). 이 결과는 식 (5.9)의 분해와 실험 관찰을 결합한 해석이다.

SwiGLU에서는 두 projection의 곱과 SiLU가 있으므로 neuron 하나를 식 (5.9)의 단순 key-value pair로 그대로 대응시키기 어렵다. 또한 superposition과 다의성 때문에 neuron보다

feature dictionary가 더 적합할 수 있지만, SAE 역시 근사 분해다. “MLP는 데이터베이스다”는 비유를 literal한 주소-값 저장 구조로 받아들이어서는 안 된다.

5.9 완전한 알고리즘을 아는 특수 사례

Tracr는 제한된 배열 처리 언어로 쓴 human-readable program을 표준 decoder-only Transformer weight로 컴파일한다. token 빈도 계산, 정렬과 괄호 검사 같은 program을 구성하며, compiler가 weight를 만들었기 때문에 source program과 모델 구조의 대응을 ground truth로 사용할 수 있다 (Lindner et al., 2023).

Tracr 모델에서는 “어떤 알고리즘을 구현하는가”를 설계 과정에서 안다. 그러나 자연어 corpus의 gradient descent로 학습한 모델에 Tracr의 알려진 program이 그대로 존재한다는 뜻은 아니다. Tracr의 가치는 다음 두 가지다.

1. 해석 방법이 알려진 component와 edge를 복원하는지 평가할 수 있다.
2. 알고리즘을 Transformer 연산으로 구현하는 구체적인 construction을 연구할 수 있다.

또 다른 완전성은 모든 floating-point 연산을 기록하는 것이다. 이는 실행 trace로서는 완전하지만, billions of scalar operations를 그대로 나열하므로 사람이 이해하는 압축된 알고리즘 설명과 다르다. 자연 학습 LLM에서 요구하는 어려운 목표는 실행 trace와 행동 사이의 충실하고 압축된 중간 설명을 찾는 일이다.

수학적 사실 또는 명시된 구현

알려진 program을 컴파일한 Transformer에는 완전한 설계 설명이 존재한다. 학습된 소형 모델에는 특정 과제의 상당한 회로 설명이 존재한다. 자연 학습한 frontier-scale LLM 전체에는 아직 이에 상응하는 전역적이고 완전한 설명이 없다.

5.10 Phase 5 학습 점검

학습 점검

1. attention-only 2-layer 모델에서 embedding에서 unembedding까지 가능한 residual path를 전개한다.
2. attention head 하나의 QK bilinear form과 OV map을 계산하고, attention weight와 direct logit contribution을 따로 시각화한다.
3. 반복 token dataset으로 previous-token head와 induction head 후보를 찾은 뒤 각 각을 ablate하여 loss 변화를 측정한다.
4. IOI 논문의 node, edge, 입력 분포, logit-difference metric과 개입 기준을 한 장의 회로 명세로 다시 작성한다.
5. Tracr program 하나를 컴파일하고 알려진 source variable과 residual direction의 대응을 blind analysis 결과와 비교한다.

한계 또는 열린 문제

회로 diagram이 그럴듯하다는 사실은 충분하지 않다. 다음 장에서는 node와 edge를 실제로 바꾸어 회로 가설이 원 모델의 반사실적 행동을 예측하는지 검사한다.

제 VI 부

Phase 6 · 인과적 기계론적 해석

6 개입으로 메커니즘을 시험하기

6.1 결론부터: 위치 찾기와 메커니즘 설명은 다르다

어떤 activation을 바꾸었을 때 출력이 달라지면 그 activation은 해당 개입과 metric 아래에서 인과적으로 관련된다. 그러나 그것만으로 그 activation이 어떤 정보를 계산했는지, 어느 upstream source에서 왔는지, 어떤 downstream path를 통해 사용되었는지는 알 수 없다. localization은 메커니즘 연구의 시작점이지 종료점이 아니다.

재현 가능한 개입에는 base input부터 aggregation까지 여섯 요소가 필요하다. 이 장에서는 이를 다음 명세로 기록한다.

$$I = (\text{base input, source input, hook, replacement, metric, aggregation}). \quad (6.1)$$

“layer 10을 patch했다”는 문장만으로는 residual input인지 output인지, 어느 position과 feature인지, 무엇으로 교체했는지가 빠져 있어 재현할 수 없다.

6.2 관찰, 상관과 개입

관찰 분포와 개입 분포는 서로 다른 질문에 답한다. 계산그래프에서 activation H 와 출력 Y 가 함께 변할 때, 관찰 분포 $p(Y | H = h)$ 는 upstream 입력이 두 변수에 함께 영향을 준 결과를 포함한다. 개입은 그래프의 해당 값을 외부에서 지정하고 나머지 downstream 계산을 다시 실행하므로 개념적으로

$$p(Y | \text{do}(H \leftarrow h')) \quad (6.2)$$

를 묻는 셈이다.

신경망의 순전파 그래프는 관측 변수 사이의 일반적인 사회과학적 인과 문제보다 직접 조작하기 쉽다. 다만 개입한 activation이 훈련 중 나타나지 않은 값이면 downstream module이 distribution 밖에서 작동한다. 따라서 인과 효과는 “원래 모델이 자연스럽게 그 상태를 만드는 과정”과 “강제로 그 상태를 넣었을 때의 반응”을 구분하여 해석해야 한다.

수학적 사실 또는 명시된 구현

activation intervention은 parameter를 다시 학습하지 않고 한 번의 실행에서 중간값을 교체한다. weight editing은 이후 모든 입력에 쓰이는 함수를 바꾼다. 두 실험은 서로 다른 인과 질문에 답한다.

6.3 ablation

ablation은 구성요소의 정상 출력을 기준값으로 바꾸거나 특정 edge를 끊고 target metric을 측정한다. component c 의 정상 실행 metric을 m_{full} , ablated 실행을 m_{-c} 라고 하면 단순 효과는

$$\Delta_c = m_{\text{full}} - m_{-c} \quad (6.3)$$

로 쓸 수 있다. metric의 방향에 따라 부호 정의를 바꿔야 한다.

중요한 것은 “제거”가 수학적으로 유일하지 않다는 점이다.

zero ablation

activation을 0으로 바꾼다. 0이 해당 hook에서 자연스러운 baseline인지 확인해야 한다.

mean ablation

reference dataset의 평균 activation으로 바꾼다. 평균을 계산한 조건에 따라 다른 정보를 보존할 수 있다.

resample ablation

다른 표본의 activation을 넣는다. marginal distribution은 더 잘 보존할 수 있지만 source 표본의 정보가 새로 들어간다.

noise ablation

선택한 noise 분포를 더하거나 대체한다. scale과 covariance가 결과를 결정한다.

head output을 0으로 만드는 것과 attention score를 $-\infty$ 로 바꾸어 특정 source를 막는 것은 다른 개입이다. 전자는 head의 모든 source contribution을 없애고, 후자는 softmax가 나머지 source에 weight를 재분배한다.

6.4 activation patching과 causal tracing

activation patching은 서로 다른 두 입력의 실행을 결합한다. clean input x_c 에서는 원하는 행동이 나타나고, corrupted input x_r 에서는 그 행동이 약해진다고 하자. 먼저 두 실행의 activation을 cache한다. 그다음 x_r 을 실행하면서 선택한 hook u 의 값을 clean activation $h_u(x_c)$ 로 교체한다.

patch recovery는 corrupted 실행이 clean activation의 이식으로 얼마나 회복되는지를 정규화한다. metric m 에 대해

$$R_u = \frac{m(x_r; h_u \leftarrow h_u(x_c)) - m(x_r)}{m(x_c) - m(x_r)} \quad (6.4)$$

로 정의할 수 있다. 분모가 작으면 불안정하며, R_u 는 0과 1 사이에 반드시 머물지 않는다. 확률, logit, logit difference와 loss 가운데 무엇을 m 으로 쓰는지에 따라 결과가 달라질 수 있다 (Zhang and Nanda, 2024; Heimersheim and Nanda, 2024).

ROME 연구의 causal tracing은 factual prompt의 subject embedding에 noise를 넣어 prediction을 손상시킨 뒤, 특정 layer와 position의 hidden activation을 clean 값으로 복원하여 사실 예측이 얼마나 회복되는지 측정했다. 이 실험은 GPT 계열 모델에서 subject token을 처리하는 middle-layer feed-forward computation의 역할을 지지했다 (Meng et al., 2022).

한계 또는 열린 문제

patch가 큰 회복을 만들었다는 사실은 해당 hook이 clean과 corrupted 조건의 차이를 운반하는 충분한 매개 지점임을 시사한다. 정보가 그 layer에서 처음 계산되었다거나 오직 그 위치에 저장되었다는 사실까지 증명하지는 않는다. residual stream은 upstream 차이를 누적해서 운반할 수 있다.

6.5 path patching과 edge intervention

node patching은 한 activation으로 들어오는 모든 upstream path를 함께 바꾼다. path 또는 edge patching은 sender s 의 clean/corrupted 차이가 특정 receiver r 에 들어가는 경로만 바꾸고, 다른 입력은 원 조건으로 유지하려고 한다. 예를 들어 head output이 뒤 head의 key input으로 가는 edge만 교체하면 value와 query 경로를 분리해 시험할 수 있다.

Transformer 구현에서는 residual stream에 여러 sender output이 더해진 뒤 receiver가 이를 읽는다. 따라서 edge를 직접 저장한 tensor가 없을 수 있다. 연구자는 receiver input을 sender별 기여로 재구성하거나 여러 차례 forward pass로 원하는 경로만 patch해야 한다. 어떤 tensor decomposition을 사용했는지가 곧 edge의 정의다.

path patching의 결과도 상호작용에 민감하다. 두 경로가 서로 대체 가능하면 각각을 따로 끊을 때 효과가 작지만 함께 끊을 때 효과가 클 수 있다. 반대로 한 경로의 효과가 다른 경로가 존재할 때만 나타날 수 있다. 따라서 single-edge effect를 단순히 합하여 전체 효과를 얻을 수 있다고 가정하면 안 된다.

6.6 interchange intervention과 causal abstraction

interchange intervention은 source example에서 얻은 내부 상태를 base example의 계산에 이식하여, 고수준 변수의 값을 바꾼 것과 같은 반사실 출력이 나오는지 검사한다. 고수준 causal model의 변수 Z 와 신경망 activation 집합 H 사이에 alignment를 제안했다고 하자. source s 의 Z 값을 base b 에 넣은 고수준 출력과, $H(s)$ 를 b 의 신경망에 넣은 저수준 출력이 일치하는지를 여러 pair에서 검사한다.

Geiger 등은 이 생각을 causal abstraction의 형식으로 정리하고, MQNLI를 위해 구성된 자연 논리 causal model의 일부 구조가 BERT 기반 모델에서 실현되는지를 interchange intervention으로 시험했다 (Geiger et al., 2021). 이 방법의 강점은 “이 neuron이 negation을 뜻한다” 같은 이름 붙이기를 넘어, 제안한 고수준 변수와 동일한 반사실 관계를 만드는지 검증한다는 데 있다.

causal abstraction은 연구자가 먼저 고수준 causal model, 변수와 alignment 후보를 정해야 한다는 한계가 있다. 시험을 통과하면 그 abstraction이 유효하다는 증거지만 유일한 abstraction이라는 뜻은 아니다. 자연어 전반에 대한 올바른 고수준 변수를 자동으로 얻는 문제도 해결되지 않는다.

6.7 attribution과 gradient 근사

first-order attribution은 작은 activation 변화의 metric 효과를 gradient 내적으로 근사한다. 미분 가능한 metric m 과 activation $\mathbf{h}$ 에 대해 변화 $\Delta\mathbf{h}$ 를 주면

$$m(\mathbf{h} + \Delta\mathbf{h}) - m(\mathbf{h}) \approx \nabla_{\mathbf{h}} m^T \Delta\mathbf{h} \quad (6.5)$$

다. attribution patching은 clean과 corrupted activation 차이를 $\Delta\mathbf{h}$ 로 두어 실제 patch를 반복하지 않고 많은 node나 edge의 효과를 근사한다.

Syed 등은 두 번의 forward pass와 한 번의 backward pass를 사용하는 이 선형 근사로 edge importance를 계산하고, 조사한 benchmark에서 기존 자동 회로 발견법보다 평균적인 circuit-recovery AUC가 높았다고 보고했다 (Syed et al., 2023). 계산량 절감은 중요하지만 근사가 큰 intervention, activation threshold와 상호작용을 정확히 포착한다는 보장은 없다.

gradient가 0에 가깝다고 구성요소가 불필요하다고 단정할 수도 없다. saturation, canceling path 또는 first-order에서 보이지 않는 곡률이 있을 수 있다. 반대로 큰 gradient는 매우 작은 국소 변화에 민감하다는 뜻이며 자연스러운 크기의 patch 효과와 같지 않을 수 있다.

6.8 자동 회로 발견

자동 회로 발견은 모든 edge를 사람이 직접 검사하는 비용을 줄이려는 방법이다. ACDC는 정한 dataset과 metric 아래에서 edge를 반복적으로 제거하고 성능 변화를 기준으로 graph를 prune한다. GPT-2 small의 greater-than 과제에서 이전 수작업 연구가 찾은 5개 component type을 모두 다시 찾고, 약 32,000개 edge 가운데 68개를 선택한 결과가 보고되었다 (Conmy et al., 2023).

자동화가 해결하는 것은 주어진 node/edge space에서 후보 subgraph를 찾는 검색 문제다. 다음 항목은 여전히 사람이 정의하거나 검증해야 한다.

- 어떤 behavior와 input distribution을 설명할 것인가?
- node와 edge를 어느 granularity로 나눌 것인가?
- 어떤 ablation baseline과 metric을 사용할 것인가?
- 선택된 구성요소를 어떤 algorithmic role로 설명할 것인가?
- 새로운 반사실 입력에서 회로가 예측을 맞히는가?

2026년의 CircuitLasso는 high-dimensional SAE feature 회로를 sparse linear regression으로 학습하여 intervention 기반 방법보다 적은 비용으로 확장하려는 접근을 제시했다. 저자들은 benchmark에서 기존 intervention 방법과 비슷한 structural accuracy를 보고했지만, 이는 2026년 6월 공개된 연구의 평가 범위에 한정된다 (Yin et al., 2026).

6.9 faithfulness, completeness, minimality

회로 C 와 전체 모델 M 을 비교할 때 세 기준을 다음처럼 구분할 수 있다.

faithfulness

회로 밖을 선택한 baseline으로 ablate했을 때 C 가 전체 모델의 target behavior를 얼마나 보존하는가?

completeness

중요한 계산이 회로 밖에 남아 있지 않은가? 회로의 complement 또는 추가 구성요소가 설명되지 않은 성능을 만들지 않는지 검사한다.

minimality

회로 안의 각 요소가 필요하거나, 제거하면 정한 기준 이상으로 설명력이 감소하는가?

이 개념에는 ablation 이후 모델을 어떻게 정의하는지가 들어간다. Miller 등은 circuit faithfulness score가 겉보기에 작은 ablation 방법 변화에도 크게 달라질 수 있음을 보고했다 (Miller et al., 2024). 따라서 숫자 하나를 회로의 내재적 속성처럼 보고하면 안 된다. 최소한 baseline distribution, patch 단위, metric, normalization과 회로 크기 곡선을 함께 제시해야 한다.

회로 평가의 권장 보고

회로 평가는 보존 성능, 제거 효과, 크기 일치 대조군과 baseline 민감도를 함께 보고해야 한다. 전체 모델, 회로만 남긴 모델, 회로를 제거한 모델과 무작위 크기 일치 회로를 같은 표에 둔다. zero, mean과 resample ablation 가운데 가능한 둘 이상을 비교한다. 개별 sample 분포와 bootstrap confidence interval을 보고하고, 회로 크기에 따른 metric curve를 제시한다.

6.10 self-repair와 방법 민감도

구성요소를 ablate하면 downstream layer는 원래 실행에서 보이지 않던 보상 update를 만들 수 있다. Hydra effect 연구는 한 attention layer를 제거했을 때 다른 layer가 일부 효과를 보상하는 사례와 late MLP가 최대가능도 token을 낮추는 counterbalancing pattern을 보고했다 (McGrath et al., 2023). 이때 ablation 후 최종 logit 변화는 제거한 구성요소의 원래 direct contribution보다 작을 수 있다.

작은 ablation 효과에는 구성요소의 작은 기여 외에도 여러 대안 설명이 있다. 구체적으로 다음 원인 가운데 하나 이상이 결과를 만들 수 있다.

- 원래 구성요소의 기여가 실제로 작다.
- 병렬 중복 경로가 같은 기능을 한다.
- ablation 때문에 downstream 보상이 새로 발생한다.
- metric이 해당 기여를 잘 드러내지 못한다.
- baseline이 원 activation과 우연히 비슷한 기능을 보존한다.

activation patching의 corruption 방식과 metric만 바꾸어도 localization map이 달라질 수 있다는 체계적 비교가 보고되었다 (Zhang and Nanda, 2024). 그러므로 한 heatmap을 메커니즘 자체처럼 해석하지 말고, 합리적인 대안 설정에서 결론이 얼마나 유지되는지 sensitivity analysis를 수행해야 한다.

6.11 Phase 6 학습 점검

학습 점검

1. clean/corrupted prompt pair를 만들고 두 조건의 차이가 목표 변수 하나에 가깝도록 검토한다.
2. residual pre, attention output, MLP output을 position과 layer별로 patch하여 logit-difference recovery map을 만든다.
3. 같은 실험을 probability와 loss metric으로 반복하고 localization 순위를 비교한다.
4. zero, dataset mean과 resample ablation을 비교하여 baseline 민감도를 기록한다.
5. node patching으로 찾은 후보를 path patching으로 좁히고, sender와 receiver의 구체적인 알고리즘 역할을 반사실 prompt로 시험한다.
6. 발견한 회로의 faithfulness, completeness와 minimality를 서로 다른 표에 보고한다.

수학적 사실 또는 명시된 구현

Phase 6의 종료 조건은 intervention 코드를 실행할 수 있다는 것이 아니다. 개입이 답하는 인과 질문, 개입이 만드는 distribution shift, metric과 baseline 의존성, 그리고 결과가 localization인지 mechanism인지 정확히 구분할 수 있어야 한다.

제 VII 부

Phase 7 · 실제 LLM 행동의 메커니즘

7 실제 LLM 행동에서 발견된 메커니즘

7.1 결론부터: 발견은 구체적이며 보편성은 미확립이다

현재 문헌은 자연 학습 LLM의 일부 행동에서 재현 가능한 내부 구조를 발견했지만, 그 결과는 대체로 특정 모델, prompt distribution, token position과 metric에 한정된다. induction, IOI와 사실 회상은 중요한 실증 사례이지만 모든 자연어 처리에 공통인 하나의 전역 알고리즘으로 합쳐지지 않았다.

행동별 연구를 읽을 때는 “무엇을 발견했는가”와 “어디까지 발견했는가”를 같은 문단에 기록해야 한다. 이 장은 각 결과 뒤에 모델, 과제, 개입과 남은 대안을 붙이며, 사람에게 익숙한 이야기만으로 빈 부분을 메우지 않는다.

7.2 factual recall

사실 회상 연구의 가장 통제된 형태는 subject–relation–attribute triple에서 다음 token을 예측하는 과제다. 예를 들어

$$(s, r, a) = (\text{Barack Obama}, \text{place of birth}, \text{Honolulu}) \quad (7.1)$$

를 “Barack Obama was born in the city of”라는 prompt로 표현하고, 모델이 attribute의 첫 token을 예측하게 한다. 이 설정은 open-ended generation보다 target logit과 token position을 명확히 정의한다.

이 과제의 정확성은 모델이 세계 사실을 추론하는 모든 방식을 포괄하지 않는다. attribute가 여러 token이면 연구가 첫 token만 설명할 수 있고, prompt paraphrase나 여러 정답이 있는 relation에서는 metric이 달라진다. 학습 corpus에 사실이 어떤 문맥과 빈도로 있었는지도 대개 완전히 알 수 없다.

ROME의 causal tracing은 GPT-2 계열에서 subject token을 처리하는 middle-layer MLP가 factual prediction에 중요한 역할을 한다는 증거를 제시했고, rank-one weight update로 선택한 factual association을 편집했다 (Meng et al., 2022). 그러나 편집 가능성과 지식의 유일한 저장 위치는 같은 명제가 아니다. 후속 분석은 localization 방법과 editing 성능이 항상 일치하지 않는다고 보고했다 (Hase et al., 2023).

7.3 factual recall의 세 단계: 원 연구의 정확한 주장

Geva 등의 원 연구가 제안한 세 단계는 “subject enrichment, relation propagation, attribute extraction”이다 (Geva et al., 2023). 연구 대상은 정답 attribute를 다음 token으로 예측한 GPT-2 XL 1.5B와 GPT-J 6B의 각각 약 1,200개 query이며, attention knockout, vocabulary projection과 component cancellation을 사용했다.

첫 단계는 early MLP가 마지막 subject position의 표현을 여러 subject-related attribute로 풍부하게 만든다는 결과다. 연구는 last-subject representation을 vocabulary에 투영한 상위 token에서 Wikipedia로 구성된 subject attribute 집합의 비율을 측정했다. GPT-2에서 early

MLP update를 취소하면 layer 40의 attribute rate가 평균 약 88% 감소했고, MHSA update를 취소한 경우에는 감소가 30% 미만이었다고 보고했다.

둘째 단계는 relation 정보가 subject 정보보다 먼저 prediction position으로 전파된다는 결과다. 연구는 last position이 non-subject position을 읽는 attention edge를 연속 layer window에서 막았을 때 나타나는 정답 확률 변화를 측정했다. 그 영향 peak가 subject-position edge를 막았을 때의 peak보다 먼저 나타났기 때문에 relation propagation으로 해석했다.

셋째 단계는 upper-layer MHSA가 relation을 담은 prediction representation으로 enriched subject를 조회하여 target attribute를 추출한다는 결과다. 연구는 last position이 subject position을 읽는 attention을 분석하고, 일부 head의 parameter가 subject-attribute mapping을 지원한다고 보고했다. 저자들은 조사한 prediction의 약 70%에서 이 extraction behavior를 관찰했다고 명시했다.

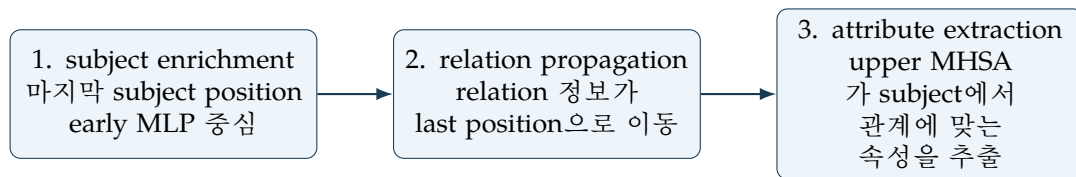


그림 7.1: Geva et al. (2023)이 제안한 사실 회상의 세 단계

이 세 단계는 software의 세 개 함수처럼 완전히 분리된 시간 구간이 아니다. layer window별 개입 효과와 projection 결과를 세 가지 기능으로 요약한 경험적 메커니즘이며, 여러 component의 계산은 겹칠 수 있다. “모델이 먼저 모든 속성을 명시적 목록으로 만들고 다음에 데이터베이스 검색을 실행한다”는 literal한 절차도 아니다.

7.3.1 2510.09033의 Section 3.3은 무엇을 다시 정의하는가

Cheang 등은 원 연구의 세 단계를 hallucination 분류를 위한 세 intervention component로 다시 기술한다 (Cheang et al., 2026). 그 논문의 표현은 early-layer subject enrichment, middle-layer subject-to-last attention flow, upper-layer last-token decoding이다. 특히 둘째 항목을 “subject information propagation”으로 부르므로, Geva 원 연구의 “relation information이 먼저 last position으로 전파된다”는 둘째 단계와 문구가 완전히 같지 않다.

Section 3.3의 문장은 token 생성 algorithm을 새로 제안하는 부분이 아니라 label을 만드는 근거를 설명하는 부분이다. 저자들은 오답을 낸 표본에서 last position이 subject token을 읽는 attention을 막고, 원 출력분포와 개입 후 출력분포의 Jensen–Shannon divergence를 계산한다. 이 값이 factual association 표본 전체의 평균보다 크면 associated hallucination, 그렇지 않으면 unassociated hallucination으로 분류한다.

수학적 사실 또는 명시된 구현

“세 단계를 거쳐 completion을 produce한다”는 말은 token 하나가 세 번 출력된다는 뜻이 아니다. 한 번의 forward pass 안에서 서로 다른 layer와 position의 activation이 subject를 풍부하게 만들고, 필요한 정보를 마지막 position으로 전달하며, 최종 logit에서 attribute token을 높이는 기능적 분해를 뜻한다.

한계 또는 열린 문제

2510.09033의 FA/AH/UH는 intervention과 threshold로 만든 조작적 범주다. 모든 hallucination이 자연적으로 이 세 ontological class 중 하나를 가진다는 증명은 아니다. 특히 AH/UH label이 subject-reliance intervention으로 정의되므로, 그 뒤의 subject-flow 차이는 독립적인 진실성 label만 분석한 결과로 읽어서는 안 된다.

7.4 hallucination과 내부 상태

Cheang 등의 핵심 결과는 조사한 두 모델에서 associated hallucination의 내부 상태가 factual association과 더 비슷하고, unassociated hallucination이 더 쉽게 분리되었다는 것이다 (Cheang et al., 2026). 분석에는 Meta-Llama-3-8B와 Mistral-7B-v0.3을 사용했고, 각 모델에서 12,293개 factual query를 분류했다.

분류 결과는 Llama에서 FA 3,506개, AH 1,406개, UH 7,381개였고, Mistral에서 각각 3,354개, 1,284개, 7,655개였다. 저자들은 subject representation patching, subject-to-last attention blocking과 last-token representation patching 뒤의 output distribution JS divergence를 layer 별로 비교했다. 이 표본에서 FA와 AH의 intervention pattern은 유사했고 UH는 다른 pattern을 보였다.

내부 표현 probe도 같은 범위에서 AH보다 UH를 더 잘 검출했다. 예를 들어 Llama의 last-token representation probe AUROC는 AH-only에서 0.69 ± 0.03 , UH-only에서 0.93 ± 0.01 이었으며, Mistral에서는 각각 0.63 ± 0.02 와 0.92 ± 0.01 이었다. 이 수치는 모든 hallucination detector의 성능 상한이 아니라 논문이 정한 dataset, split, feature와 label에서의 결과다.

이 연구가 지지하는 제한된 해석은 내부 signal이 진실성 자체보다 parametric association 사용 여부를 더 강하게 반영할 수 있다는 것이다. AH는 강하지만 잘못된 association을 통해 정답 회상과 비슷한 경로를 탈 수 있으므로, confidence나 hidden state만으로 참과 거짓을 안정적으로 가르기 어렵다는 설명이다.

한계 또는 열린 문제

probe 실패는 진실성 정보가 모델 내부 어디에도 존재하지 않는다는 증명이 아니다. 선택한 layer, position, 선형 feature와 dataset에서 충분히 복원되지 않았다는 결과다. 반대로 UH 검출 성공도 모델이 명시적인 “나는 모른다” 변수를 사용한다는 증명은 아니다.

7.5 in-context learning과 copying

in-context learning(ICL)은 parameter update 없이 현재 context의 example이나 pattern에 따라 다음-token behavior가 달라지는 현상이다. induction head는 반복 pattern의 복사라는 한 메커니즘을 제공하며, 소형 attention-only 모델에서는 인과적으로 자세히 분석되었다 (Olsson et al., 2022).

일부 이론·실험 연구는 Transformer가 context 안의 linear regression example에서 gradient descent와 유사한 update를 구현할 수 있음을 보였다 (von Oswald et al., 2023; Akyürek et al., 2023). 이 결과는 특정 synthetic regression task와 architecture에서 입출력 함수를 분석한 것이다. 자연어 ICL 전체가 실제 parameter gradient descent를 문자 그대로 실행한다는 결론은 아니다.

copying도 단일 기능이 아니다. token을 그대로 반복하는 회로, 앞 token 다음의 token을 복사하는 induction, 이름 후보를 prediction position으로 옮기는 name mover와 긴 span을 재생하는 행동은 서로 다른 QK 조건과 OV 내용을 가질 수 있다. “attention은 복사한다”는 문장은 source selection, value transformation과 target logit 효과를 분해해야 검증 가능하다.

7.6 reasoning과 chain-of-thought

chain-of-thought(CoT)는 모델이 중간 자연어 단계를 출력하도록 prompting하는 외부 생성 형식이다. Wei 등은 충분히 큰 모델의 여러 arithmetic, commonsense와 symbolic reasoning benchmark에서 few-shot CoT prompt가 standard prompting보다 성능을 높인 결과를 보고했다 (Wei et al., 2022). 이 결과는 출력된 문장이 내부 계산의 완전한 transcript임을 보이지 않는다.

CoT의 비충실성은 입력의 biasing feature가 답을 바꾸면서도 생성된 explanation에는 그 영향이 언급되지 않는 실험으로 관찰되었다 (Turpin et al., 2023). 따라서 “모델이 이유를 말했으므로 실제로 그 이유로 답했다”는 추론은 성립하지 않는다. CoT는 행동 데이터이자 추가 계산을 유도하는 scratchpad가 될 수 있지만, 내부 메커니즘의 ground truth label은 아니다.

내부 reasoning 메커니즘을 주장하려면 중간 variable과 causal path를 별도로 시험해야 한다. 예를 들어 두 단계 질의에서 중간 entity direction을 찾고 이를 교체했을 때 최종 answer가 예측대로 바뀌는지 검사할 수 있다. 정답률 상승, probe와 intervention은 서로 다른 증거이므로 하나의 “reasoning 능력” 점수로 합치지 않는다.

7.7 entity tracking, planning과 refusal

entity tracking은 같은 이름이나 대상을 문맥의 여러 position에서 구분하고 해당 정보를 prediction position에 유지하는 계산을 요구한다. IOI 회로는 duplicate token과 이름 억제, name-moving head가 협력하는 한 구체적 사례를 제공하지만, 임의 길이 대화의 모든 entity state를 설명하지는 않는다 (Wang et al., 2022).

2025년 Anthropic의 attribution-graph 사례 연구는 Claude 3.5 Haiku에서 multi-hop reasoning, 시의 운율 단어를 미리 고르는 planning, familiar entity와 unfamiliar entity의 구분, harmful request와 refusal에 관한 feature 경로를 보고했다 (Lindsey et al., 2025). 저자들은 feature direction 개입과 후속 perturbation으로 선택한 그래프 가설을 검증했다.

이 결과는 frontier production model 내부에서 사람이 해석할 수 있는 중간 계산이 존재할 수 있다는 existence proof다. 그러나 저자들은 만족스러운 insight를 얻은 prompt가 시도한 prompt의 약 4분의 1이었고, 성공 사례에서도 모델 메커니즘의 작은 일부만 포착했으며, 선택한 사례가 도구의 한계에 의해 편향되었다고 명시했다.

거절에 관한 feature가 발견되었다고 모델의 안전 정책 전체가 한 방향에 저장된 것은 아니다. pretraining과 post-training feature의 상호작용, prompt-specific graph와 여러 competing path가 존재할 수 있다. 발견된 방향을 steering했을 때 behavior가 바뀌는 결과도 원래 입력에서 그 방향이 모든 거절을 충분히 설명한다는 주장과 구분해야 한다.

7.8 대형 모델의 attribution graph

attribution graph 방법은 원 모델 자체를 곧바로 완전히 분해하지 않고, MLP를 더 해석하기 쉬운 cross-layer transcoder(CLT)로 바꾼 replacement model을 사용한다 (Ameisen et al., 2025).

CLT feature는 한 layer의 residual stream을 읽고 현재와 뒤 MLP layer의 출력을 재구성하며, sparsity penalty와 reconstruction loss로 학습된다.

local replacement model은 한 prompt에서 원 attention pattern과 normalization denominator를 고정하고 CLT reconstruction error를 더하여 원 activation과 logit을 일치시킨다. 이 조정은 그 prompt의 forward value를 맞추지만, perturbation에 대한 반응까지 원 모델과 같다는 보장은 없다. 저자들은 이 차이를 mechanistic faithfulness로 따로 평가한다.

attribution graph의 edge는 고정된 비선형 조건 아래 feature 사이의 선형 효과를 나타낸다. graph에는 token embedding, 활성화 feature, error node와 output logit이 들어가며, 출력에 대한 기여가 작은 node와 edge를 pruning한다. 이 구조는 한 prompt를 읽기 쉬운 graph로 줄이지만, pruning threshold 아래의 다수 경로와 attention pattern을 만든 QK 메커니즘을 놓칠 수 있다.

replacement model의 근사 오차는 현재 방법의 핵심 한계다. methods paper에서 가장 큰 18-layer CLT replacement model은 선택된 pretraining-style prompt 집합의 약 50%에서 원 모델과 같은 top next token을 냈다. local error correction으로 한 prompt의 수치를 맞춰도 동일한 내부 메커니즘을 사용한다는 결과가 자동으로 따라오지 않는다.

제한된 해석 · 현재 attribution graph의 올바른 지위

attribution graph는 원 모델에 대한 구체적이고 검증 가능한 메커니즘 가설을 생성하는 강력한 현미경이다. 그것은 원 모델의 모든 계산을 손실 없이 사람이 읽을 수 있는 graph로 변환한 완전한 해부도가 아니다.

7.9 모델 간 일반화 문제

같은 decoder-only Transformer 계열도 factual recall의 중요한 구성요소가 같다고 가정할 수 없다. Choe 등은 GPT, Llama, Qwen과 DeepSeek 계열을 비교하고, 조사한 Qwen-based 모델에서는 이전 GPT-style pattern과 달리 earliest layer attention module의 기여가 MLP보다 크다는 결과를 보고했다 (Choe et al., 2025).

모델 간 차이는 architecture 이름 하나로 설명되지 않는다. normalization, GQA, tokenizer와 block 배치뿐 아니라 학습 데이터, objective와 seed가 다르다. 같은 기능을 다른 parameter 경로가 구현할 수도 있고, 표면적으로 비슷한 head가 다른 representation basis를 사용할 수도 있다.

일반화를 주장하려면 여러 수준을 분리해 측정해야 한다. behavioral generalization은 같은 정답을 내는지 묻고, representational generalization은 alignment 뒤 유사한 feature가 있는지 묻고, mechanistic generalization은 같은 causal dependency가 intervention으로 유지되는지 묻는다. 첫째가 같아도 둘째와 셋째가 자동으로 같아지지 않는다.

2026년의 LM-agent 기반 circuit explanation 연구도 자동 설명의 범위를 명확히 보여준다. HyVE는 84개 semi-synthetic Transformer circuit과 163개 component annotation으로 만든 benchmark에서 관찰, 가설, causal validation을 반복했고, Llama-3-8B arithmetic case study도 제시했다. 그러나 어떤 backbone도 일관되게 최선이 아니었고 validation 단계의 실패가 주요 장애로 남았다고 보고했다 (Khan et al., 2026).

7.10 Phase 7 학습 점검

학습 점검

1. Geva et al. (2023)의 세 단계 각각에 대해 model, position, layer 범위, intervention, metric과 수치를 원문 표로 복원한다.
2. 2510.09033의 FA/AH/UH labeling code를 재현하고 JS threshold가 바뀔 때 표본 구성이 얼마나 달라지는지 sensitivity curve를 그린다.
3. factual recall 결과를 paraphrase, multi-token attribute와 다른 checkpoint에서 반복하고 실패 사례를 별도 표로 기록한다.
4. CoT 문장과 내부 causal variable이 일치하는지 answer-preserving, rationale-changing counterfactual로 시험한다.
5. attribution graph의 reconstruction error node, pruning threshold와 perturbation validation을 함께 보고한다.
6. behavior, representation과 mechanism의 모델 간 일반화를 각각 다른 metric으로 평가한다.

수학적 사실 또는 명시된 구현

Phase 7의 종료 조건은 유명한 회로 이름을 암기하는 것이 아니다. 각 논문의 결론을 원 실험 범위로 되돌리고, 관찰된 단계와 연구자가 붙인 해석을 구분하며, 새로운 모델에서 무엇을 다시 검증해야 하는지 설계할 수 있어야 한다.

제 VIII 부

현재의 경계와 연구 프로그램

8 완전한 설명은 어디까지 왔는가

8.1 현재의 결론

2026년 8월 23일 현재, 자연어로 학습한 frontier-scale LLM 전체의 내부 알고리즘을 완전하고 충실하며 사람이 이해할 수 있는 형태로 발견한 연구는 없다. 현재 연구는 특정 행동의 부분 회로, 특정 prompt의 attribution graph, 일부 feature와 개입 가능한 방향을 제공하며, 논문들도 전역적 완전성을 주장하지 않는다.

반면 고정된 구현의 수치적 순전과는 완전히 명세할 수 있다. tokenizer, weights, input IDs, dtype, kernel, random state와 decoding rule을 고정하면 embedding에서 모든 Q/K/V, residual, MLP, logit과 token 선택까지 재현할 수 있다. 이 의미의 “계산을 안다”와 학습된 의미 알고리즘을 안다는 주장을 혼동해서는 안 된다.

완전한 알고리즘을 아는 Transformer는 설계된 특수 사례에 존재한다. Tracr처럼 알려진 program을 compiler가 weight로 옮긴 모델은 source algorithm과 component 대응을 갖는다 (Lindner et al., 2023). 이 결과는 자연 학습 LLM을 완전히 해석했다는 결과가 아니라, 완전한 ground truth를 가진 실험실을 제공한다.

8.2 완전성, 충실성, 압축성과 확장성

유용한 완전한 설명은 적어도 네 축을 동시에 만족해야 한다. 한 축에서 좋은 결과를 다른 축의 증거로 대체할 수 없으므로, 각 축을 별도 metric과 실패 조건으로 관리해야 한다.

완전성

target behavior에 필요한 계산을 빠뜨리지 않고 포함하는가? 선택한 분포 밖의 행동까지 주장하려면 그 분포도 별도로 시험해야 한다.

인과적 충실성

설명 모델과 원 모델이 관측 출력뿐 아니라 관련 intervention과 counterfactual에 같은 방향과 크기로 반응하는가?

압축성과 인간 이해 가능성

scalar operation의 전체 trace보다 짧으면서도 새로운 입력과 개입 결과를 예측할 수 있는 변수, 절차와 불변량을 제공하는가?

확장성

소형 모델의 한 과제에 그치지 않고 더 큰 모델, 긴 context, 여러 언어와 행동에 적용할 수 있으며 계산비용과 사람 검토 비용이 감당 가능한가?

이 네 축에는 tension이 존재한다. 모든 multiplication을 기록하면 완전성은 높지만 설명은 압축되지 않으며, head 몇 개에 짧은 이름을 붙이면 이해하기 쉽지만 누락된 feature와 경로가 많을 수 있다. replacement model은 해석 가능한 단위를 늘리지만 reconstruction error와 mechanistic mismatch를 도입할 수 있다 (Ameisen et al., 2025).

설명 of 유일성도 보장되지 않는다. Méloux 등은 Boolean function과 소형 MLP의 후보 설명을 열거한 실험에서 같은 행동을 재현하는 여러 회로, 한 회로의 여러 해석과 같은 algorithm의 여러 subspace alignment가 존재하는 비식별성 사례를 보였다 (Méloux et al., 2025). 따라서 “유일한 진짜 이야기”보다 반사실 예측력, 조작 가능성, 간결성과 범위를 명시한 여러 동등 설명을 평가해야 할 수 있다.

8.3 알려진 것의 경계

현재 지식의 경계는 “아는가, 모르는가”의 이분법보다 설명 수준별로 기록하는 편이 정확하다. 표 8.1는 이 책의 결론을 단계별로 요약한다.

표 8.1: 현대 decoder-only LLM에 대해 알려진 것과 남은 문제

수준	강하게 알려진 내용	해결되지 않은 내용
구현과 수학	공개 코드와 checkpoint의 순전파, tensor shape, logits와 decoding을 재현할 수 있다.	비공개 training data와 recipe, hardware별 bit-wise 차이는 접근권한과 기록에 달려 있다.
표현	많은 변수는 특정 activation에서 probe되며 일부 방향은 steering할 수 있다.	자연스러운 계산 단위의 유일한 dictionary와 전역 의미는 확립되지 않았다.
국소 인과	ablation, patching과 editing으로 특정 input과 behavior의 구성요소 효과를 측정할 수 있다.	baseline, metric, self-repair와 distribution shift에 강건한 표준은 발견 중이다.
회로	induction, IOI, factual recall과 여러 사례에서 구성요소 협력이 발견되었다.	행동 전반을 덮는 완전하고 모델 간에 안정적인 회로 지도는 없다.
대형 모델	feature 기반 attribution graph로 선택한 prompt의 중간 계산을 부분적으로 추적할 수 있다.	replacement error, attention QK, graph pruning과 성공 사례 편향이 남는다.
전역 알고리즘	설계·검과 일한 소형 Transformer에는 ground-truth program이 존재한다.	자연 학습 frontier LLM 전체의 충실하고 압축된 algorithmic description은 없다.

“모른다”는 문장은 아무것도 발견하지 못했다는 뜻이 아니다. induction head와 IOI, subject enrichment와 attribute extraction, causal abstraction, SAE feature와 attribution graph는 구체적인 진전이다. 다만 발견 범위를 보존해야 이 조각들을 잘못된 보편 이론으로 조기에 굳히지 않을 수 있다.

8.4 연구 프로그램

완전한 설명을 향한 실용적 연구 프로그램은 ground truth가 있는 작은 모델에서 평가 기준을 확립한 뒤 자연 모델과 더 큰 행동으로 확장해야 한다. 다음 여섯 단계는 이 책의 Phase 1부터 7까지를 연구 작업으로 변환한다.

1. 실행을 고정한다. 공개 model과 tokenizer의 순전파를 직접 구현하고 모든 hook의 tensor를 reference code와 비교한다.
2. 행동을 좁게 정의한다. dataset, prompt generator, target token과 metric을 versioned artifact로 만든다.
3. 표현 가설을 만든다. probe, geometry와 SAE로 후보 variable을 찾되, 이 단계의 결론을 decodability로 제한한다.
4. 경로 가설을 만든다. QK/OV, DLA와 자동 검색으로 sender, receiver와 intermediate variable을 명시한다.
5. 반사실로 반증을 시도한다. activation과 path intervention, causal abstraction, adversarial prompt와 model transfer로 예측을 시험한다.
6. 설명을 압축하고 감사한다. graph가 보존하는 행동, reconstruction error, 누락 경로, 사람 해석 일치도와 계산비용을 함께 보고한다.

각 단계의 산출물은 논문 문장보다 실행 가능한 기록이어야 한다. commit hash, checkpoint digest, token IDs, hook name, seed, dtype, raw metric과 실패 사례를 보존하면 설명이 바뀔 때 어떤 근거가 수정되었는지 추적할 수 있다.

첫 번째 장기 project는 “한 token을 완전히 추적하기”가 적합하다. 소형 공개 LLM의 한 factual prompt에서 embedding부터 target logit까지 모든 activation과 direct contribution을 저장하고, Geva의 세 단계가 paraphrase와 counterfactual에서 유지되는지 검증한다.

두 번째 장기 project는 “알려진 program을 blind하게 재발견하기”가 적합하다. Tracr 모델의 source program을 가린 상태에서 probe, patching, ACDC와 SAE로 회로를 찾고, ground truth와 정밀도와 재현율을 비교한다. 이 project는 자연 LLM에서 정답이 없는 평가를 시작하기 전에 방법의 false positive와 false negative를 측정하게 해 준다.

8.5 종료 조건이 아니라 갱신 규칙

이 책은 “LLM을 완전히 이해했다”는 종료 선언보다 근거가 들어올 때 설명을 고치는 갱신 규칙을 제공한다. 각 주장은 source, model, dataset, intervention, metric과 반증 조건을 가지며, 더 강한 결과가 나오면 claim ledger의 범위를 넓히거나 기존 문장을 철회한다.

새 논문은 headline이 아니라 증거 사슬로 편입해야 한다. 공개 구현과 data가 있는지, baseline에 강건한지, 독립 model에서 재현되는지, replacement model의 오차가 포함되는지, 결과가 correlation인지 causation인지 확인한 뒤 해당 상자 등급을 정한다.

완전한 이해라는 목표는 현재의 불완전성을 숨기지 않을 때만 과학적 연구 계획이 된다. 우리가 완전히 아는 순전파부터 시작하여 표현, 국소 인과, 회로와 행동 일반화를 차례로 검증하면, 아직 존재하지 않는 전역 설명을 이미 발견한 것처럼 말하지 않으면서도 그 목표를 향해 누적적으로 전진할 수 있다.

수학적 사실 또는 명시된 구현

계산 자체는 재현 가능하고, 학습된 메커니즘은 부분적으로만 해석되었으며, 완전한 설명은 열린 연구 문제다. 앞으로의 진전은 더 많은 그럴듯한 이름이 아니라 더 강한 반사실 예측, 더 작은 설명 격차와 더 넓은 재현 범위로 측정해야 한다.

제 IX 부

부록

A 기호와 용어의 기준

A.1 기호를 고정하는 이유

기호의 일관성은 Transformer의 source와 destination 축을 뒤바꾸는 오류를 막는 첫 번째 검산 장치다. 이 책은 수식에서 column vector를 기본으로 쓰되, batch 계산의 tensor는 마지막 축을 feature로 두는 일반적인 코드 convention을 함께 표시한다.

표 A.1: 핵심 기호와 차원

기호	의미	값 또는 shape
B	batch 크기	양의 정수
T	현재 token sequence 길이	양의 정수
V	vocabulary 크기	양의 정수
L	Transformer block 수	양의 정수
d	residual stream 차원	d_{model}
d_h	한 attention head의 차원	모델에 따라 d/H_q 인 경우가 흔함
d_{ff}	MLP 중간 차원	모델별 양의 정수
H_q	query head 수	MHA에서는 보통 $H_q = H_{kv}$
H_{kv}	key/value head 수	MQA에서는 1, GQA에서는 $1 < H_{kv} < H_q$
t_i	position i 의 token ID	$\{0, \dots, V - 1\}$
E	token embedding matrix	$R^{V \times d}$
$X^{(\ell)}$	block ℓ 입구 residual stream	$R^{B \times T \times d}$
Q, K, V	query, key, value tensor	head 축을 분리하면 각각 $[B, H, T, d_h]$
A^h	head h 의 attention weight	$[B, T, T]$, 행은 destination, 열은 source
W_U	unembedding 또는 LM-head weight	$R^{V \times d}$
Z	vocabulary logits	$R^{B \times T \times V}$
θ	모델의 모든 학습 parameter	vector 또는 구조화된 tensor 집합
L	학습 loss	scalar
Δx_c	구성요소 c 가 쓰는 residual update	R^d
m	해석 실험의 target metric	logit difference, probability, loss 등

A.2 축과 index의 방향

attention matrix는 destination-by-source 순서로 정의한다. 즉 $A_{i,j}$ 는 destination position i 가 source position j 의 value를 읽는 weight다. causal decoder에서는 $j > i$ 인 미래 source가 mask된다.

행렬의 전치는 vector convention 때문에 달라질 수 있다. column vector를 쓰면 $\mathbf{q} = \mathbf{W}_Q^T \mathbf{x}$ 로, row-vector tensor를 쓰는 PyTorch식 표기에서는 $\mathbf{Q} = \mathbf{XW}_Q$ 로 적을 수 있다. 두 식의 parameter 저장 shape가 같다는 가정 없이 기계적으로 비교해서는 안 된다.

layer 번호는 문헌마다 0부터 또는 1부터 시작한다. 이 책의 수식은 embedding 뒤 residual을 $\mathbf{X}^{(0)}$ 으로 두고, block $\ell \in \{0, \dots, L-1\}$ 가 $\mathbf{X}^{(\ell+1)}$ 을 만든다. 원 논문 수치를 옮길 때는 논문의 번호 convention을 그대로 표시한다.

A.3 한국어와 원어 용어

용어는 정착된 한국어가 자연스러우면 번역하고, 코드와 논문 검색에 필요한 원어는 첫 등장이거나 표에서 함께 제시한다. 억지 번역으로 구현 이름을 흐리지 않기 위해 attention, logit, embedding, probe와 checkpoint처럼 국내 기술 문헌에서 원어가 널리 쓰이는 용어는 원어를 유지한다.

표 A.2: 권장 용어와 이 책에서의 의미

표기	대응 원어	이 책에서의 의미
순전파	forward pass	고정된 입력과 parameter에서 activation과 출력을 계산하는 과정
역전파	backpropagation	chain rule로 loss의 parameter gradient를 계산하는 방법
잔차 스트림	residual stream	각 sublayer가 update를 읽고 더하는 position별 d 차원 상태
활성값	activation	특정 입력에서 중간 node가 실제로 취한 수치
가중치·parameter	weight, parameter	학습되어 여러 입력에 공통으로 쓰이는 값
특징·feature	feature	의미 변수 또는 학습된 dictionary element이며 문맥별 정의가 필요함
회로	circuit	지정한 행동을 설명한다고 가정한 계산 부분그래프
개입	intervention	activation, edge 또는 weight를 외부에서 바꾸고 결과를 재계산하는 실험
복원 가능성	decodability	별도 readout이 내부 상태에서 변수를 예측할 수 있는 성질

표기	대응 원어	이 책에서의 의미
인과적 충실성	causal faithfulness	설명과 원 모델이 관련 개입에 같은 반응을 보이는 정도
사실 회상	factual recall	subject와 relation에서 attribute token을 예측하는 행동
환각	hallucination	평가 기준상 factual claim이 근거 또는 사실과 맞지 않는 출력

A.4 인식론적 동사의 강도

동사의 선택은 증거 수준을 표시한다. “계산한다”와 “증명한다”는 정의나 수학적 귀결에, “관찰했다”와 “보고했다”는 특정 실험 결과에, “시사한다”와 “해석할 수 있다”는 대안이 남는 설명에 사용한다.

“모델이 안다”, “생각한다”와 “기억한다”는 조작적 정의 없이 사용하지 않는다. 예를 들어 “모델이 Honolulu를 안다” 대신 “이 prompt에서 Honolulu token의 logit이 가장 높다” 또는 “paraphrase 집합에서 정답을 생성했다”라고 쓴다. anthropomorphic 표현이 필요한 경우에는 관찰 가능한 행동을 바로 뒤에 정의한다.

학습 점검

논문을 읽을 때 모든 핵심 동사에 밑줄을 긋고 “정의”, “관찰”, “개입”, “해석” 또는 “추측”으로 분류한다. 저자의 문장도 실험 설계보다 강하면 그대로 사실 상자에 옮기지 않는다.

B 핵심 수식 유도

B.1 softmax의 shift invariance와 안정적 계산

softmax는 모든 logit에 같은 상수를 더해도 변하지 않는다. 임의의 $c \in R$ 에 대해

$$\frac{e^{z_i+c}}{\sum_j e^{z_j+c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (\text{B.1})$$

가 성립하기 때문이다.

안정적인 softmax는 가장 큰 logit을 빼서 overflow를 줄인다. $m = \max_j z_j$ 로 두면

$$p_i = \frac{e^{z_i-m}}{\sum_j e^{z_j-m}} \quad (\text{B.2})$$

이고 모든 지수의 입력이 0 이하가 된다. 이 변환은 위의 shift invariance 때문에 확률을 바꾸지 않는다.

softmax의 Jacobian은 vocabulary logit 사이의 결합을 보여 준다. 미분하면

$$\frac{\partial p_i}{\partial z_j} = p_i(\mathbf{1}[i = j] - p_j) \quad (\text{B.3})$$

를 얻는다. 따라서 token j 의 logit 변화는 j 의 확률뿐 아니라 다른 모든 token의 확률에도 영향을 준다.

B.2 cross-entropy와 logit gradient

one-hot target y 에 대한 softmax cross-entropy의 logit gradient는 $\mathbf{p} - \mathbf{y}$ 다. target index를 k 라고 하면

$$L = -\log p_k = -z_k + \log \sum_j e^{z_j}, \quad (\text{B.4})$$

$$\frac{\partial L}{\partial z_i} = -\mathbf{1}[i = k] + \frac{e^{z_i}}{\sum_j e^{z_j}} = p_i - y_i. \quad (\text{B.5})$$

이 식은 정답 logit을 올리고 현재 확률에 비례하여 다른 logit을 내리는 gradient 신호를 보여 준다.

LM head의 weight gradient는 hidden state와 logit error의 outer product다. row-vector convention에서 $\mathbf{z} = \mathbf{W}_U \mathbf{h} + \mathbf{b}$ 라면

$$\frac{\partial L}{\partial \mathbf{W}_U} = (\mathbf{p} - \mathbf{y}) \mathbf{h}^\top, \quad \frac{\partial L}{\partial \mathbf{h}} = \mathbf{W}_U^\top (\mathbf{p} - \mathbf{y}). \quad (\text{B.6})$$

이 gradient는 chain rule을 통해 앞 block과 embedding까지 전달된다.

B.3 scaled dot-product attention의 차원과 scale

attention score matrix의 두 T 축은 destination과 source position에서 생긴다. batch와 head 축을 생략하면

$$\mathbf{Q} \in R^{T \times d_h}, \quad \mathbf{K}^T \in R^{d_h \times T}, \quad \mathbf{Q}\mathbf{K}^T \in R^{T \times T} \quad (\text{B.7})$$

이며, $\mathbf{AV}$ 는 $(T \times T)(T \times d_h) = T \times d_h$ 가 된다.

$1/\sqrt{d_h}$ scale은 독립이고 평균 0, 분산 1인 query/key coordinate라는 이상화에서 dot-product 분산을 차원과 무관하게 만든다. $q_r k_r$ 들이 독립이고 분산 1이면

$$\text{Var}\left(\sum_{r=1}^{d_h} q_r k_r\right) = d_h, \quad \text{Var}\left(\frac{1}{\sqrt{d_h}} \sum_{r=1}^{d_h} q_r k_r\right) = 1. \quad (\text{B.8})$$

실제 학습된 q 와 k 는 이 독립 가정을 정확히 만족하지 않지만, 유도는 scale의 설계 동기를 설명한다 (Vaswani et al., 2017).

causal mask는 softmax 뒤가 아니라 score에 적용하는 것이 정확한 정의다. 미래 score를 $-\infty$ 로 두면 그 지수는 0이 되고 허용된 source만 다시 정규화된다. softmax 뒤의 weight를 0으로 만들고 재정규화하지 않으면 row sum이 1이 아니므로 다른 함수가 된다.

B.4 여러 head의 출력은 residual update의 합이다

concatenation 뒤의 output projection은 head별 output projection의 합으로 분해된다. $\mathbf{O}_{\text{cat}} = [\mathbf{O}^1 \dots \mathbf{O}^H]$ 이고 $\mathbf{W}_O$ 를 대응하는 row block으로

$$\mathbf{W}_O = \begin{bmatrix} \mathbf{W}_O^1 \\ \vdots \\ \mathbf{W}_O^H \end{bmatrix} \quad (\text{B.9})$$

라고 나누면 block multiplication으로

$$\mathbf{O}_{\text{cat}} \mathbf{W}_O = \sum_{h=1}^H \mathbf{O}^h \mathbf{W}_O^h \quad (\text{B.10})$$

를 얻는다. 이 등식은 head output을 residual 방향으로 분석하는 근거다.

head별 합은 head가 기능적으로 독립이라는 뜻이 아니다. 모든 head의 Q/K/V가 같은 residual input을 읽고, 뒤 layer가 합쳐진 결과를 비선형적으로 처리하므로 한 head의 효과는 다른 head와 경로에 의존할 수 있다.

B.5 RoPE가 상대 offset을 포함하는 이유

2차원 회전행렬은 $\mathbf{R}(a)^T \mathbf{R}(b) = \mathbf{R}(b-a)$ 를 만족한다. 삼각함수의 덧셈정리를 회전행렬 곱에 적용하면 바로 확인할 수 있다. 따라서

$$(\mathbf{R}(i)\mathbf{q})^T (\mathbf{R}(j)\mathbf{k}) = \mathbf{q}^T \mathbf{R}(j-i)\mathbf{k} \quad (\text{B.11})$$

이고 score가 절대 position의 차이 $j-i$ 에 의존한다.

RoPE의 상대성은 content vector와 offset의 상호작용이다. 같은 offset이라도 q 와 k 가 다르면 score가 다르며, RoPE만으로 특정 distance를 선택하는 attention pattern이 보장되지는 않는다. frequency schedule과 학습된 projection이 함께 작동한다.

B.6 RMSNorm이 보존하고 바꾸는 양

gain과 ϵ 을 잠시 제외한 RMS normalization은 출력의 root-mean-square를 1로 만든다. $r(\mathbf{x}) = \sqrt{d^{-1} \sum_i x_i^2}$ 라면

$$r\left(\frac{\mathbf{x}}{r(\mathbf{x})}\right) = 1 \quad (\text{B.12})$$

이다. 실제 식에서는 $\epsilon > 0$ 과 coordinate-wise gain g 때문에 최종 output의 RMS가 정확히 1일 필요는 없다.

RMSNorm은 입력의 전체 positive scale에 거의 불변이다. $\epsilon = 0$ 이고 $a > 0$ 이면

$$\text{RMSNorm}(a\mathbf{x}) = g \odot \frac{a\mathbf{x}}{ar(\mathbf{x})} = \text{RMSNorm}(\mathbf{x}). \quad (\text{B.13})$$

$a < 0$ 이면 방향 부호가 바뀌며, 유한 ϵ 에서는 정확한 scale invariance가 아니다.

pre-norm residual stream의 norm은 layer를 거치며 계속 변할 수 있다. RMSNorm은 sub-layer가 읽는 사본을 정규화할 뿐, identity path의 residual state를 정규화된 값으로 교체하지 않기 때문이다.

B.7 KV cache가 같은 attention을 계산하는 이유

KV cache는 과거 position의 key와 value가 새 token 때문에 바뀌지 않는 causal decoder에서 projection을 재사용한다. 새 position $T + 1$ 의 hidden state는 과거 position의 hidden state를 바꾸지 않으므로, 같은 layer의 $\mathbf{K}_{1:T}$ 와 $\mathbf{V}_{1:T}$ 도 이전 prefill/decode 때 계산한 값과 같다.

cache와 전체 재계산의 새-position attention row는 이상적인 산술에서 동일하다. 두 방법 모두 같은 q_{T+1} 과 연결된 $[\mathbf{K}_{1:T}; \mathbf{k}_{T+1}]$, $[\mathbf{V}_{1:T}; \mathbf{v}_{T+1}]$ 를 사용하기 때문이다. 과거 query row는 다음-token logit에 필요하지 않으므로 다시 만들지 않는다.

KV cache의 원소 수는 batch, layer, KV head, sequence, head dimension과 K/V 두 종류의 곱이다. padding과 metadata를 제외한 단순 원소 수는

$$N_{\text{cache}} = 2BLH_{kv}Td_h \quad (\text{B.14})$$

이며, element당 byte 수를 곱하면 기본 저장량을 얻는다. tensor parallel 복제, alignment, quantization과 paging은 실제 메모리를 바꾼다.

B.8 GQA가 줄이는 parameter와 cache

GQA는 query projection보다 key/value projection의 output dimension을 줄인다. bias를 제외하고 input dimension이 d 이면

$$N_Q = dH_qd_h, \quad (\text{B.15})$$

$$N_K + N_V = 2dH_{kv}d_h \quad (\text{B.16})$$

이다. MHA의 $H_{kv} = H_q$ 에서 GQA의 작은 H_{kv} 로 바꾸면 K/V parameter와 cache가 같은 비율로 줄지만 Q와 output projection은 이 식에서 줄지 않는다.

GQA의 계산량 감소는 단계에 따라 다르다. K/V projection과 cache traffic은 줄지만 각 query head는 여전히 attention weight와 value sum을 계산한다. kernel과 sequence 길이를 고려하지 않고 전체 inference FLOPs가 H_{kv}/H_q 배로 줄었다고 말하면 부정확하다.

B.9 direct logit attribution과 final norm

final normalization이 없으면 residual update의 logit 기여는 정확히 선형이다. target token a 와 contrast token b 의 unembedding 차이를 $w_{ab} = w_{U,a} - w_{U,b}$ 라고 하면

$$z_a - z_b = w_{ab}^T \left(x^{(0)} + \sum_c \Delta x_c \right) + (b_a - b_b) \quad (B.17)$$

이므로 각 c 의 항은 $w_{ab}^T \Delta x_c$ 다.

final normalization이 있으면 구성요소 기여는 일반적으로 서로 결합된다. 예를 들어 RM-SNorm의 denominator는 모든 residual component의 합으로 결정되므로 한 update를 바꾸면 다른 component가 받는 공통 scale도 바뀐다. 연구자는 normalization을 실제로 적용한 component-wise projection, frozen-scale approximation 또는 local Jacobian 가운데 무엇을 사용했는지 밝혀야 한다.

B.10 activation patching의 first-order 근사

attribution patching은 finite intervention을 Taylor 전개 of 첫 항으로 근사한다. corrupted activation을 h_r , clean activation을 h_c 라고 하면

$$m(h_c) - m(h_r) = \nabla m(h_r)^T (h_c - h_r) + O(\|h_c - h_r\|^2) \quad (B.18)$$

다. 차이가 작고 곡률이 약할수록 first-order score가 실제 patch effect에 가까울 수 있다.

큰 patch와 activation threshold는 선형 근사의 오차를 키울 수 있다. ReLU나 gate의 on/off가 바뀌거나 attention softmax가 크게 이동하면 고차항이 중요해진다. 계산량을 줄이기 위해 근사를 사용한 뒤 상위 후보에는 실제 patch를 수행하는 이유가 여기에 있다.

B.11 Jensen–Shannon divergence

Jensen–Shannon(JS) divergence는 두 확률분포의 대칭적인 차이를 측정한다. 분포 P, Q 와 $M = (P + Q)/2$ 에 대해

$$JS(P, Q) = \frac{1}{2} KL(P \| M) + \frac{1}{2} KL(Q \| M) \quad (B.19)$$

로 정의한다. 자연로그를 쓰면 값은 0과 $\log 2$ 사이에 있으며, 로그 base 2를 쓰면 상한은 1이다.

JS divergence는 어떤 token의 logit이 변했는지를 혼자 알려 주지 않는다. vocabulary 전체 분포의 변화를 요약하므로 target token과 무관한 mass 이동도 값에 들어간다. 사실 회상 개입에서는 target logit difference와 JS divergence를 함께 보면 질문이 다른 두 metric을 구분할 수 있다.

학습 점검

각 유도는 작은 무작위 tensor를 double precision으로 계산하여 등식을 검증한다. 수학적 등식의 허용 오차와 GPU mixed-precision kernel의 허용 오차를 따로 정하며, shape와 축을 assertion으로 고정한다.

C 재현 실험 프로토콜

C.1 재현의 최소 단위

해석 실험은 그림이 아니라 실행 조건과 raw 결과가 함께 있어야 재현된다. 모든 실험은 다음 manifest를 저장한다.

- repository commit과 변경 여부;
- model repository, revision과 weight file digest;
- tokenizer repository, revision, chat template와 실제 token IDs;
- framework, CUDA 또는 Metal, kernel, device와 dtype;
- random seed, deterministic 설정과 batch 순서;
- dataset source, license, sample ID와 필터링 규칙;
- hook 이름, tensor shape와 저장 시점;
- intervention, baseline, metric과 aggregation code;
- 표에 쓰기 전 sample-level raw value와 실패 log.

artifact directory는 실험 입력과 결과를 덮어쓰지 않도록 run ID별로 분리한다. 다음 구조는 필수 규칙이 아니라 권장 예시이다.

Listing C.1: 권장 실험 artifact 구조

```
runs/<run-id>/
  manifest.json
  config.yaml
  tokenized_inputs.jsonl
  metrics.parquet
  interventions.parquet
  figures/
  logs/
  environment.txt
```

C.2 Lab 1: 손으로 계산한 attention과 코드의 일치

첫 실험의 목표는 길이 3, head dimension 2인 causal attention을 한 숫자도 건너뛰지 않고 재현하는 것이다. 정수 또는 소수 한 자리의 Q, K, V를 직접 정하고 score, scale, mask, row-wise softmax와 weighted sum을 종이에 계산한다.

성공 기준은 독립 구현 두 개가 같은 결과를 내는 것이다. 하나는 명시적인 for-loop로, 다른 하나는 matrix multiplication으로 작성하고 double precision에서 각 원소 오차를 10^{-10} 이하로 확인한다. mask를 softmax 전과 후에 잘못 적용한 negative test도 넣어 차이를 기록한다.

C.3 Lab 2: 최소 decoder-only Transformer 구현

두 번째 실험의 목표는 high-level Transformer module을 호출하지 않고 한 block의 모든 tensor를 직접 만드는 것이다. 최소 구현에는 token embedding, RMSNorm, RoPE, causal MHA 또는 GQA, SwiGLU, residual addition, final norm과 LM head가 포함되어야 한다.

성공 기준은 모든 중간 shape와 causal invariant가 assertion을 통과하는 것이다. 미래 token ID만 바꾼 두 sequence를 넣었을 때 그 이전 position의 logits가 허용 오차 안에서 같아야 한다. 각 attention row의 합, masked weight, RMS denominator와 final vocabulary dimension도 검사한다.

hook 출력은 parameter와 activation을 분리하여 두 종류의 값을 혼동하지 않게 해야 한다. 다음 의사코드는 최소 shape trace를 보여 준다.

Listing C.2: 한 block의 shape 추적 의사코드

```
x = embed(token_ids)           # [B, T, d]
r = rmsnorm_attn(x)            # [B, T, d]
q, k, v = project_and_split(r) # [B, Hq/Hkv, T, dh]
q, k = apply_rope(q, k, positions)
a = causal_softmax(q @ k.transpose(-1, -2) / sqrt(dh))
x = x + merge_heads(a @ repeat_kv(v)) @ W_o
r = rmsnorm_mlp(x)
x = x + (silu(r @ W_gate) * (r @ W_up)) @ W_down
logits = final_norm(x) @ W_unembed.T # [B, T, V]
```

C.4 Lab 3: 공개 reference implementation과 대조

세 번째 실험의 목표는 직접 구현한 block을 실제 공개 checkpoint의 reference code와 대조하는 것이다. 같은 weight를 load하고 tokenizer output, embedding, 각 normalization, Q/K/V, RoPE 뒤 tensor, attention output, MLP output, residual과 logits를 hook한다.

성공 기준은 dtype별 허용 오차와 첫 불일치 지점을 자동 보고하는 것이다. 마지막 logit만 비슷하다고 통과시키지 않으며, layer별 최대 절대오차와 상대오차를 저장한다. fused kernel로 중간값을 직접 꺼낼 수 없으면 unfused reference path와 비교한다.

불일치의 흔한 원인은 transposed weight, RoPE의 even/odd packing, GQA의 head mapping, RMSNorm ϵ , bias, position offset와 tokenizer special token이다. 원인을 고친 뒤에는 해당 오류를 재현하는 regression test를 남긴다.

C.5 Lab 4: prefill과 KV-cache decode의 동등성

네 번째 실험의 목표는 full recomputation과 cached decode가 같은 next-token distribution을 계산하는지 검증하는 것이다. prompt의 마지막 logit을 full pass로 얻고, token을 하나씩 넣는 cached path의 각 step logit과 비교한다.

성공 기준은 동일 dtype과 deterministic kernel에서 정한 오차 안에 있고 greedy token이 모든 step에서 일치하는 것이다. cache의 layer, KV head, sequence와 head-dimension 축을 기록하고, RoPE position이 cache length만큼 offset되는지 검사한다.

C.6 Lab 5: probe는 정보 존재만 보고한다

다섯 번째 실험의 목표는 representation에서 label을 복원하는 증거와 모델 사용의 증거를 분리하는 것이다. factual prompt의 last-subject와 last-position residual을 layer별로 모아 relation 또는 정답 class에 대한 regularized linear probe를 학습한다.

성공 기준은 held-out entity split, random-label control과 majority baseline을 포함하는 것이다. 같은 entity가 train과 test의 paraphrase에 동시에 들어가지 않도록 split 단위를 entity로 잡는다. probe accuracy, calibration, selectivity와 confidence interval을 함께 보고한다.

후속 intervention은 probe direction이 원 모델 행동에 미치는 효과를 따로 시험한다. direction을 더하고 빼는 scale sweep에서 target logit, KL divergence, unrelated-task degradation과 norm 변화를 기록한다. 효과가 있어도 자연 입력에서 원 모델이 같은 방향을 사용한다는 결론은 path-level 검증 전까지 보류한다.

C.7 Lab 6: induction circuit 재현

여섯 번째 실험의 목표는 반복 token pattern에서 induction score와 인과 효과를 함께 측정하는 것이다. 무작위 token 열의 앞부분을 뒤에서 반복하고, 현재 token과 같은 token의 바로 다음 position에 attention하는 pattern을 head별로 계산한다.

성공 기준은 pattern score가 높은 head의 ablation이 반복 구간의 next-token loss를 선택적으로 악화시키는지 확인하는 것이다. 비반복 control 구간, 크기 일치 random head와 previous-token head 후보를 함께 비교한다. 큰 attention pattern만으로 induction head label을 확정하지 않는다.

C.8 Lab 7: activation patching의 방법 민감도

일곱 번째 실험의 목표는 clean/corrupted pair에서 localization map이 metric과 corruption에 얼마나 의존하는지 측정하는 것이다. subject 또는 relation 하나만 바꾼 factual prompt pair를 만들고 residual pre, attention output과 MLP output을 position-layer별로 patch한다.

성공 기준은 최소 세 metric과 세 baseline의 순위 상관을 보고하는 것이다. target logit, logit difference와 probability를 비교하고, zero, mean과 resample replacement를 비교한다. sample 별 recovery가 분모가 작은 경우를 별도 표시하고, 평균 heatmap만 보고하지 않는다.

C.9 Lab 8: 사실 회상의 세 단계 재현

여덟 번째 실험의 목표는 Geva 등의 세 단계를 같은 모델에서 재현한 뒤 architecture transfer를 시험하는 것이다. 첫 단계에서는 last-subject projection의 attribute rate와 early MLP cancellation을, 둘째 단계에서는 relation-to-last attention knockout을, 셋째 단계에서는 subject-to-last upper attention과 attribute logit 기여를 측정한다.

성공 기준은 원 논문의 GPT-2 XL 또는 GPT-J 조건을 먼저 재현하고, Llama/Qwen 계열의 결과를 별도 실험으로 보고하는 것이다. layer 번호는 절대값과 전체 깊이에 대한 비율을 함께 기록하며, 다른 architecture에서 실패해도 원 결과의 재현 실패와 generalization 실패를 구분한다.

C.10 Lab 9: 2510.09033의 분류와 검출 결과 감사

아홉 번째 실험의 목표는 FA/AH/UH label이 subject-reliance threshold에 얼마나 민감한지 검증하는 것이다. greedy BF16 생성, factual correctness 판정, subject-to-last attention blocking, 원 분포와 개입 분포의 JS divergence, FA 평균 threshold의 순서로 원 절차를 재현한다 (Cheang et al., 2026).

성공 기준은 threshold sweep과 독립 label audit를 포함하는 것이다. FA 평균뿐 아니라 quantile과 held-out calibration threshold를 비교하고, AH/UH 표본 이동률을 보고한다. probe train/test split은 entity와 template leakage를 막으며, correctness judge의 일부 표본은 사람이 blind review한다.

원 연구의 conclusion과 label construction의 결합을 따로 검사한다. AH/UH가 subject-flow intervention으로 정의되므로 같은 flow metric에서 나타나는 차이는 부분적으로 예상된다. 새로운 검증에는 label에 직접 쓰지 않은 intervention, paraphrase robustness와 외부 factual verification을 추가한다.

C.11 Lab 10: ground-truth circuit에서 방법 평가

열 번째 실험의 목표는 자연 모델에 적용하기 전에 해석 도구의 탐지 오류를 알려진 program에서 측정하는 것이다. Tracr로 token frequency, sorting 또는 parenthesis checking program을 컴파일하고 source code를 보지 않는 분석 단계와 ground-truth 비교 단계를 분리한다.

성공 기준은 component와 edge recovery의 precision, recall, faithfulness와 description length를 함께 보고하는 것이다. probe, activation patching, ACDC, attribution patching과 SAE를 같은 node granularity에서 비교한다. 알고리즘 이름을 맞추는 것뿐 아니라 unseen input과 intervention output을 예측하는지도 평가한다.

C.12 실험 중단과 실패 보고 규칙

실패한 재현은 조건을 충분히 기록하면 유효한 결과다. model revision, token mismatch, memory 부족, numerical divergence, 불안정한 metric과 논문에 없는 구현 세부사항을 실패 유형으로 나누어 남긴다.

결론을 바꾸는 분석 선택은 결과를 본 뒤 조용히 수정하지 않는다. threshold, layer window, metric과 sample filter를 사전에 config에 저장하고, 사후 탐색은 exploratory analysis로 표시한다. 여러 선택을 시험했다면 성공한 설정만 골라 보고하지 않고 전체 sensitivity를 제시한다.

공통 통과 기준

실험은 다른 사람이 새 환경에서 manifest만으로 실행할 수 있고, raw sample-level 결과에서 책의 표와 그림을 다시 만들 수 있으며, negative control과 적어도 하나의 합리적인 대안 설정에서 핵심 결론이 유지될 때 통과한다.

D 주장-증거 감사표

D.1 감사표의 목적

주장-증거 감사표는 핵심 문장이 인용 범위를 넘어가는 일을 막는다. 각 행은 책의 요약 주장, 증거 종류, 적용 범위와 결론을 바꿀 수 있는 조건을 함께 기록한다.

표 D.1: 핵심 주장과 증거 범위

ID	핵심 주장	근거	범위와 갱신 조건
C01	고정된 weight와 실행 조건에서 Transformer 순전파는 입력 token IDs를 logits로 보내는 명시적 함수다.	architecture 수식과 공개 코드 (Vaswani et al., 2017; Meta AI, 2024)	sampling 난수와 비결정적 kernel은 별도다. 코드가 바뀌면 구현 주장을 갱신한다.
C02	residual block은 현재 state에 attention과 MLP update를 더한다.	대수적 architecture 정의 (Vaswani et al., 2017; Elhage et al., 2021)	pre/post-norm과 parallel block은 식이 다르므로 checkpoint별로 제한한다.
C03	attention weight만으로 output의 인과 원인을 식별할 수 없다.	QK/OV 분해와 상반된 설명 실험 (Jain and Wallace, 2019; Wiegrefe and Pinter, 2019)	weight와 value-output effect, intervention을 함께 측정하면 더 강한 주장이 가능하다.
C04	RMSNorm은 평균을 빼지 않고 root-mean-square로 scale을 정규화한다.	정의와 원 논문 (Zhang and Senrich, 2019)	ϵ , gain, bias와 적용 위치는 구현별로 확인한다.
C05	RoPE를 적용한 query-key 내적은 상대 offset을 대수적으로 포함한다.	회전행렬 등식 (Su et al., 2024)	훈련 길이 밖의 품질 일반화는 이 등식으로 보장되지 않는다.
C06	GQA는 여러 query head가 더 적은 KV head를 공유하여 KV cache를 줄인다.	architecture와 실험 (Ainslie et al., 2023)	품질과 실제 속도는 model, kernel과 hardware에 따라 재평가한다.
C07	FlashAttention은 근사 attention이 아니라 I/O-aware exact attention algorithm으로 제안되었다.	algorithm과 복잡도 분석 (Dao et al., 2022)	exact는 실수식 기준이며 bit-wise equality는 연산 순서와 dtype에 의존한다.
C08	linear probe 성공은 정보의 복원 가능성을 보이지만 원 모델의 사용을 자동으로 보이지 않는다.	probe control 분석 (Hewitt and Liang, 2019)	causal intervention과 path evidence가 추가되면 사용 주장을 강화할 수 있다.
C09	tuned lens는 중간 표현을 읽기 위해 layer별 translator를 학습하는 진단 모델이다.	방법 정의 (Belrose et al., 2023)	translator는 원 Transformer의 구성요소가 아니다.

ID	핵심 주장	근거	범위와 갱신 조건
C10	toy model은 sparsity 조건에서 중첩과 다의적 neuron이 나타날 수 있음을 구성적으로 보인다.	통제된 toy-model 실험 (Elhage et al., 2022)	자연 LLM의 모든 다의성이 같은 원인이라는 결론은 보류한다.
C11	SAE는 activation을 희소 dictionary로 근사 분해하지만 의미 feature의 유일성을 보장하지 않는다.	SAE 목적 함수와 benchmark (Bricken et al., 2023; Karvonen et al., 2025)	reconstruction, sparsity, disentanglement와 intervention을 함께 평가한다.
C12	induction head 는 $[A][B] \dots [A] \rightarrow [B]$ pattern을 구현하는 확인된 회로 motif다.	소형 모델의 인과 분석과 대형 모델의 간접 증거 (Olsson et al., 2022)	모든 자연어 ICL의 충분한 설명이라는 주장은 하지 않는다.
C13	GPT-2 small의 IOI 행동에는 26개 attention head, 7개 기능군의 회로 설명이 제시되었다.	causal intervention과 회로 평가 (Wang et al., 2022)	해당 IOI distribution과 model에 한정하며 저자도 남은 격차를 보고했다.
C14	Tracr-compiled Transformer는 source program이 알려져 있어 ground-truth 해석 평가에 쓸 수 있다.	compiler construction (Lindner et al., 2023)	자연 corpus에서 학습된 모델의 알고리즘을 발견한 결과는 아니다.
C15	activation patching 결과는 corruption, metric과 replacement 선택에 민감할 수 있다.	체계적 비교와 방법 지침 (Zhang and Nanda, 2024; Heimersheim and Nanda, 2024)	여러 합리적 설정에서 sensitivity analysis를 보고한다.
C16	ablation 뒤 downstream component가 제거 효과를 일부 보상할 수 있다.	언어 모델의 self-repair 실험 (McGrath et al., 2023)	모델과 component별 효과가 다르므로 모든 작은 ablation effect를 보상으로 해석하지 않는다.
C17	circuit faithfulness score는 ablation 방법에 강하게 의존할 수 있다.	여러 방법의 robustness 비교 (Miller et al., 2024)	baseline과 circuit-size curve를 함께 보고하며 단일 score를 절대값으로 취급하지 않는다.
C18	Geva 등의 factual recall 설명은 subject enrichment, relation propagation과 upper-MHSA attribute extraction의 세 단계다.	GPT-2 XL과 GPT-J의 약 1,200개 정답 query 분석 (Geva et al., 2023)	원 연구의 model, correct-prediction filter와 first-token 과제에 한정한다.
C19	2510.09033에서는 AH가 FA와 유사하고 UH가 더 쉽게 분리되는 내부 pattern이 보고되었다.	Llama-3-8B와 Mistral-7B-v0.3의 intervention과 probe (Cheang et al., 2026)	AH/UH는 subject-reliance JS threshold로 만든 조작적 label이며 다른 hallucination taxonomy로 자동 일반화하지 않는다.
C20	attribution graph는 대형 모델의 선택된 prompt에서 해석 가능한 중간 경로를 부분적으로 드러낼 수 있다.	CLT replacement model, graph와 perturbation (Ameisen et al., 2025; Lindsey et al., 2025)	replacement error, frozen non-linearity, pruning과 성공 사례 편향 때문에 완전한 원 모델 graph가 아니다.
C21	factual recall mechanism은 autoregressive Transformer architecture 사이에서 다를 수 있다.	GPT, Llama, Qwen과 DeepSeek 계열 비교 (Choe et al., 2025)	한 비교 연구의 model set과 방법에 한정하며 독립 재현으로 갱신한다.

ID	핵심 주장	근거	범위와 갱신 조건
C22	자연 학습 frontier LLM 전체의 완전하고 충실하며 압축된 알고리즘 설명은 아직 없다.	현재 방법 논문의 명시적 한계와 case-study 범위 (Ameisen et al., 2025; Lindsey et al., 2025)	전역 coverage와 intervention faithfulness를 입증한 새 연구가 나오면 즉시 갱신한다.
C23	같은 행동을 만족하는 기계론적 설명은 유일하지 않을 수 있다.	Boolean function과 소형 MLP의 완전 열거 사례 (Méloux et al., 2025)	자연 LLM의 모든 설명이 비식별이라는 정리는 아니며, uniqueness 조건 연구에 따라 갱신한다.
C24	CoT는 내부 계산의 충실한 transcript로 가정할 수 없다.	biasing feature와 explanation의 불일치 실험 (Turpin et al., 2023)	model과 과제별 faithfulness를 별도로 평가하며 모든 CoT가 비충실하다고 일반화하지 않는다.

D.2 갱신 절차

새 근거는 기존 행을 삭제하기보다 version history와 함께 갱신한다. 결과가 재현되지 않으면 모델·데이터·구현 차이인지 원 결론의 불안정성인지 구분하고, 범위 열을 먼저 좁힌다.

주장의 등급은 근거가 강해질 때만 올린다. probe에서 intervention으로, 단일 prompt에서 held-out distribution으로, 단일 model에서 architecture transfer로 증거가 확장되면 해당 범위를 명시한다. 반대 결과가 나오면 평균적인 결론 속에 숨기지 않고 별도 행이나 예외로 기록한다.

학습 점검

책의 새 판을 만들기 전에는 본문의 모든 \citep와 \citet가 감사표 또는 비핵심 배경 주장으로 분류되는지 검사한다. C22처럼 부재를 주장하는 문장은 최신 문헌 검색과 주요 방법 논문의 limitation을 다시 확인한 뒤 날짜를 갱신한다.

참고문헌

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://arxiv.org/abs/2305.13245>.
- Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? investigations with linear models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2211.15661>.
- Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, et al. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018. URL <https://arxiv.org/abs/1502.05767>.
- Nora Belrose, Zach Furman, Logan Smith, Danny Halawi, Igor Ostrovsky, Lev McKinney, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens, 2023. URL <https://arxiv.org/abs/2303.08112>. arXiv:2303.08112.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nicholas Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- Chi Seng Cheang, Hou Pong Chan, Wenxuan Zhang, and Yang Deng. Do LLMs really know what they don't know? internal states mainly reflect knowledge recall rather than truthfulness, 2026. URL <https://arxiv.org/abs/2510.09033>. arXiv:2510.09033v3.
- Minyeong Choe, Haehyun Cho, Changho Seo, and Hyunil Kim. Do all autoregressive transformers remember facts the same way? a cross-architecture analysis of recall mechanisms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. URL <https://arxiv.org/abs/2509.08778>.
- Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2304.14997>.

- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://arxiv.org/abs/2205.14135>.
- Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020. URL <https://mml-book.github.io/>.
- Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. URL <https://transformer-circuits.pub/2021/framework/index.html>.
- Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *Transformer Circuits Thread*, 2022. URL https://transformer-circuits.pub/2022/toy_model/index.html.
- Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstractions of neural networks. In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL <https://arxiv.org/abs/2106.02997>.
- Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021. URL <https://arxiv.org/abs/2012.14913>.
- Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. Dissecting recall of factual associations in auto-regressive language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://arxiv.org/abs/2304.14767>.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun. Does localization inform editing? surprising differences in causality-based localization vs. knowledge editing in language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2301.04213>.
- Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching, 2024. URL <https://arxiv.org/abs/2404.15255>. arXiv:2404.15255.
- John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 2733–2743, 2019. URL <https://aclanthology.org/D19-1275/>.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al.

- Training compute-optimal large language models. *Advances in Neural Information Processing Systems*, 35, 2022. URL <https://arxiv.org/abs/2203.15556>.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020. URL <https://arxiv.org/abs/1904.09751>.
- Sarthak Jain and Byron C. Wallace. Attention is not explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3543–3556, 2019. URL <https://aclanthology.org/N19-1357/>.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>. arXiv:2001.08361.
- Adam Karvonen, Can Rager, Johnny Lin, Curt Tigges, Joseph Bloom, David Chanin, Yeu-Tong Lau, Eoin Farrell, Callum McDougall, Kola Ayonrinde, Demian Till, Matthew Wearden, Arthur Conmy, Samuel Marks, and Neel Nanda. SAEBench: A comprehensive benchmark for sparse autoencoders in language model interpretability. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://arxiv.org/abs/2503.09532>.
- Ayan Antik Khan, Harsh Kohli, Yuekun Yao, Huan Sun, and Ziyu Yao. Can language model agents be helpful circuit explainers in mechanistic interpretability?, 2026. URL <https://arxiv.org/abs/2606.24026>. arXiv:2606.24026.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018. URL <https://aclanthology.org/D18-2012/>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL <https://arxiv.org/abs/2309.06180>.
- David Lindner, János Kramár, Sebastian Farquhar, Matthew Rahtz, Thomas McGrath, and Vladimir Mikulik. Tracr: Compiled transformers as a laboratory for interpretability. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2301.05062>.
- Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, et al. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.

- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Thomas McGrath, Matthew Rahtz, János Kramár, Vladimir Mikulik, and Shane Legg. The hydra effect: Emergent self-repair in language model computations, 2023. URL <https://arxiv.org/abs/2307.15771>. arXiv:2307.15771.
- Maxime Méloux, Silviu Maniu, François Portet, and Maxime Peyrard. Everything, everywhere, all at once: Is mechanistic interpretability identifiable? In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2502.20914>.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://arxiv.org/abs/2202.05262>.
- Meta AI. Llama 3 reference implementation: model.py. GitHub repository, 2024. URL <https://github.com/meta-llama/llama3/blob/main/llama/model.py>. Accessed 2026-08-23.
- Joseph Miller, Bilal Chughtai, and William Saunders. Transformer circuit faithfulness metrics are not robust. In *Conference on Language Modeling*, 2024. URL <https://arxiv.org/abs/2407.08734>.
- Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads, 2022. URL <https://arxiv.org/abs/2209.11895>. arXiv:2209.11895; Transformer Circuits Thread.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://arxiv.org/abs/2203.02155>.
- Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2311.03658>.
- Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, pages 157–163, 2017. URL <https://aclanthology.org/E17-2025/>.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2305.18290>.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986. URL <https://www.nature.com/articles/323533a0>.

- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>. arXiv:1707.06347.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725, 2016. URL <https://aclanthology.org/P16-1162/>.
- Noam Shazeer. Fast transformer decoding: One write-head is all you need, 2019. URL <https://arxiv.org/abs/1911.02150>. arXiv:1911.02150.
- Noam Shazeer. GLU variants improve transformer, 2020. URL <https://arxiv.org/abs/2002.05202>. arXiv:2002.05202.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. URL <https://arxiv.org/abs/2104.09864>.
- Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery, 2023. URL <https://arxiv.org/abs/2310.10348>. arXiv:2310.10348.
- Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, et al. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. URL <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2305.04388>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://arxiv.org/abs/1706.03762>.
- Johannes von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers learn in-context by gradient descent. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. URL <https://arxiv.org/abs/2212.07677>.
- Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: A circuit for indirect object identification in GPT-2 Small, 2022. URL <https://arxiv.org/abs/2211.00593>. arXiv:2211.00593.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models.

- In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://arxiv.org/abs/2201.11903>.
- Sarah Wiegrefe and Yuval Pinter. Attention is not not explanation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 11–20, 2019. URL <https://aclanthology.org/D19-1002/>.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, 2020. URL <https://arxiv.org/abs/2002.04745>.
- Naiyu Yin, Dennis Wei, Tian Gao, Amit Dhurandhar, Karthikeyan Natesan Ramamurthy, and Yue Yu. Scalable circuit learning for interpreting large language models, 2026. URL <https://arxiv.org/abs/2606.16939>. arXiv:2606.16939; Mechanistic Interpretability Workshop at ICML 2026.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019. URL <https://arxiv.org/abs/1910.07467>.
- Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2309.16042>.

찾아보기

절제 (ablation), 38
활성값 패칭 (activation patching), 38
기여도 그래프 (attribution graph), 48
자기회귀 생성 (autoregressive generation),
9

역전파 (backpropagation), 5
기저 (basis), 2

인과 추상화 (causal abstraction), 39
인과적 어텐션 (causal attention), 11
사고 연쇄 (chain-of-thought), 47
회로 (circuit), 30

사실 회상 (factual recall), 44
충실성 (faithfulness), 40
FlashAttention, 20

그룹 질의 어텐션 (GQA), 18

환각 (hallucination), 46
은닉 상태 (hidden state), 2

induction head, 32
간접 목적어 식별 (IOI), 33

KV 캐시, 19

logit lens, 26

MLP, 12

OV 회로, 32

경로 패칭 (path patching), 39
prefill과 decode, 19
선형 probe, 25

QK 회로, 32
query-key-value (QKV), 11

잔차 스트림 (residual stream), 2
RMSNorm, 17
RoPE, 17

자기 복구 (self-repair), 41
softmax, 5
희소 오토인코더 (SAE), 27
중첩 (superposition), 26
SwiGLU, 18

토큰나이저 (tokenizer), 19
Tracr, 34